BOSTON UNIVERSITY

COLLEGE OF ENGINEERING

Thesis

ADAPTIVE FEEDBACK CONTROL OF A MULTI-INPUT MICROBIAL FUEL CELL USING OBSERVER-BASED LINEAR QUADRATIC REGULATION

by

JONATHAN D. MILLER

B.S., Xavier University, 2023

Submitted in partial fulfillment of the

Boston University requirements for the degree of

Master of Science in Engineering (MSE)

2026

Approved by

First Reader

Linda H. Doerrer, Ph.D.
Associate Professor of Chemistry

Second Reader

David A. Castanon, Ph.D.
Professor of Electrical and Computer Engineering

Third Reader

Ioannis Ch. Paschalidis, Ph.D.
Distinguished Professor of Electrical and Computer Engineering

For everyone who believed this was possible,

and for those who believed in me.

Acknowledgments

I would like to express my sincere gratitude to my advisor, Professor Linda H. Doerrer, for her mentorship, encouragement, and unwavering support throughout this project. Her willingness to supervise a thesis that crossed disciplinary boundaries into control systems engineering was instrumental in allowing this research to take shape, and her advocacy consistently opened doors that made the work possible.

I am deeply grateful to Professor David Castañon for his rigorous and sustained engagement with the control systems design and mathematical formulation of this thesis. Over the past six months, his weekly guidance substantially strengthened the state-space modeling, observer design, and LQR implementation presented here. I also thank Professor Ioannis Paschalidis for serving on my committee and for the additional perspective he brought to the technical work.

Most of all, I am especially grateful to Dr. Ioannis Ieropoulos of the University of Southampton, whose foundational contributions to microbial fuel cell research shaped the direction of this project. His generosity with time, encouragement, and sustained engagement and patience over the past several years have had a lasting influence on both this work and my development as a researcher.

I thank the staff of the Boston University Engineering Product Innovation Center (EPIC) for their assistance with kiln firing of the ceramic membranes and access to fabrication equipment. I would also like to thank The Boston University Department of Electrical and Computer Engineering for supporting this research.

Finally, I am grateful to my family and friends for their patience and encouragement throughout the demands of graduate school. This work would not have been possible without their support.

ADAPTIVE FEEDBACK CONTROL OF A MULTI-INPUT MICROBIAL FUEL CELL USING OBSERVER-BASED LINEAR QUADRATIC REGULATION

JONATHAN D. MILLER

Boston University, College of Engineering, MSE 2026

Major Professor: Linda H. Doerrer, Ph.D.

Associate Professor of Chemistry

ABSTRACT

Microbial fuel cells (MFCs) are a promising nascent technology for simultaneous wastewater treatment and renewable electricity generation, yet their practical development is constrained by the difficulty of maintaining stable biochemical conditions. The variable feedstock inputs and necessary recurring perturbations create a complex and chaotic system resistant to basic control methodology. This thesis addresses those constraints by developing and experimentally validating a linear quadratic regulator (LQR) control framework for MFCs. We present evidence that not only is the system controllable in theory, but furthermore that a physics-based closed-loop feedback control system for a multi-input microbial fuel cell is indeed able to elevate and sustain increased MFC voltage and power output over time.

A three-state discrete-time state-space model was identified from experimental data, capturing the coupled dynamics of volatile fatty acid (VFA) concentration, electrical conductivity (EC), and acidity/alkalinity (pH) in an 875 mL reactor that was periodically stirred and fed by controlled injections of apple juice and wastewater (water-diluted fermented urine) as manipulated inputs. System identification was carried out using Differential Evolution followed by Nelder-Mead optimization refinement, minimizing one-step-ahead prediction error on experimental sensor time series. A discrete-time Kalman filter observer was used to estimate the unmeasured VFA state from EC and pH measurements at every 5-second control step. For the controller model, a discrete-time Linear Quadratic Regulator (LQR) was designed to minimize a (quadratic) cost function that uses weighted penalties for a multi-dimensional state deviation from setpoint (set equal to the operating point for linearization for robust simplicity). Control inputs were computed at 60-second intervals and delivered by peristaltic pumps controlled by an Arduino GIGA R1 WiFi microcontroller (firmware versions 21 and 22).

Four experiments compared LQR adaptive dosing against a matched open-loop baseline under repeated acid perturbations to pH 6.5. In Experiment 1, conducted under fully standardized starting conditions, the LQR controller produced 30.3 percent greater integrated MFC energy output over matched 90-minute windows. Furthermore, it maintained oxidation-reduction potential (approximately 45 mV greater magnitude than the baseline) in the negative direction, indicating a more strongly reducing environment favorable to exoelectrogenic activity. These results demonstrate that not only is the environment controllable in both theory and practice, but further that observer-based LQR control produces measurable improvements in bio-electrochemical output from a multi-

input MFC system, establishing a control systems foundation for the practical deployment of MFC technology in decentralized wastewater treatment applications.

Contents

Acknowledgments v

Abstract vi

List of Tables x

List of Figures xi

List of Abbreviations xii

1 Introduction and Literature Review 1

1.1 Climate Change and Energy Consumption 1

1.2 The Global Wastewater Crisis 2

1.3 Microbial Fuel Cell Fundamentals 4

1.4 Historical Development of Microbial Fuel Cells 6

1.5 Advances in Power Density Through Miniaturization 7

1.6 Long-Term Performance and Scalability 9

1.7 Key Biochemical Parameters Affecting MFC Performance 11

1.8 Microbial Fuel Cell Control Systems 15

1.9 Research Gap and Objectives 17

2 System Design 19

2.1 Manufacturing Methods 19

2.2 MFC Assembly 21

2.3 Reactor Construction 22

2.4 Instrumentation 24

3 Methodology 26

3.1 Experimental Setup 26

3.2 Sensors and Calibration 27

3.3 State-Space Model 28

3.4 A Matrix Identification 31

3.5 B Matrix Identification 34

3.6 LQR Controller Design 37

3.7 Kalman Filter Observer 40

3.8 Experiment Protocol 43

3.9 Data Analysis 45

4 Results 46

4.1 Overview 46

4.2 Open-Loop Simulation 47

4.3 Closed-Loop Simulation 49

4.4 Experiment 1 55

4.5 Experiment 2 59

4.6 Experiment 3 62

4.8 Experimental Timing and Phase Structure 64

4.9 Statistical Hypothesis Testing 65

5 Conclusion 68

5.1 Summary 68

5.2 Key Findings 69

5.3 Limitations 71

5.4 Future Work 72

5.5 Broader Impact 74

References 76

Appendix A: Arduino GIGA R1 WiFi Firmware (v22) 83

Appendix B: Python Analysis Code 95

List of Tables

3.1 Sensor calibration parameters 27

3.2 Continuous-time A matrix (A_c) 32

3.3 Discrete-time A matrix (A_d) 33

3.4 Open-loop system eigenvalues 33

3.5 Discrete-time B matrix (B_d) 35

3.6 Observation matrix H 36

3.7 LQR design matrices Q and R 38

3.8 LQR gain matrix K 39

3.9 Closed-loop eigenvalues 39

3.10 Kalman filter noise covariance matrices Q_proc and R_meas 41

3.11 Steady-state Kalman gain matrix K 41

3.12 Kalman filter observer eigenvalues 42

4.1 Open-loop simulation summary 48

4.2 Closed-loop simulation summary 50

4.3 Steady-state control inputs in closed-loop simulation 51

4.4 Experiment 1 acid phase endpoint verification 55

4.5 Experiment 1 MFC energy output by phase 56

4.6 Experiment 1 mean sensor values by phase 57

4.7 Experiment 2 run configuration 59

4.8 Experiment 2 results summary 61

4.9 Experiment 3 results summary 63

4.11 Experiment run windows 65
4.12 Cross-experiment Welch's t-test results 66

List of Figures

2.1 Ceramic membrane tubes after firing 20

2.2 Assembled MFC inside reactor, side view 22

2.3 Assembled MFC reactor, top view showing sensor ports and cathode assembly 23

3.1 Control system block diagram 29

3.2 Sensor data used for B matrix identification 35

3.3 Full experimental setup showing fluid reservoirs, pumps, reactor, and control electronics 26

4.1 Open-loop simulation: pH, EC, and VFA trajectories 48

4.2 Closed-loop simulation: LQR vs. baseline 50

4.3 Experiment 1: Integrated MFC energy output comparison, LQR vs. baseline 56

4.5 Experiment 1 simulation: 120-minute LQR vs. baseline pH, EC, and VFA trajectories 60

4.6 Experiment 1: MFC voltage and ORP comparison, LQR vs. ratio-matched baseline 62

Glossary of Abbreviations and Acroynms

Abbreviations and acronyms are listed in alphabetical order for reference.

Abbreviation/ Acronym	Definition
ADC	Analog-to-Digital Converter
COD	Chemical Oxygen Demand
CSTR	Continuously Stirred Tank Reactor
ARE	(Continuous) Algebraic Riccati Equation
DARE	Discrete Algebraic Riccati Equation
EC	Electrical Conductivity
LQR	Linear Quadratic Regulator
MFC	Microbial Fuel Cell
MIMO	Multiple-Input Multiple-Output
ORP	Oxidation-Reduction Potential
PEM	Proton Exchange Membrane
SISO	Single-Input Single-Output
VFA	Volatile Fatty Acid
WHO	World Health Organization

Notation: Control Model Matrices and Vectors

All matrices and vectors are defined in discrete time with sample period T_s = 5 seconds unless otherwise noted. Dimensions are given as rows × columns.

Symbol	Name	Definition
`A_d`	Discrete-time state matrix	3×3 matrix encoding autonomous dynamics of VFA, EC, and pH over one 5-second sample period. Diagonal elements represent self-decay; off-diagonal elements represent cross-coupling between states.
`B_d`	Discrete-time input matrix	3×2 matrix mapping pump dose inputs (urine and apple juice, in mL/min) to state increments per 5-second step. Columns correspond to urine and apple juice respectively.
`C`	Observation matrix	2×3 matrix selecting the measured outputs (EC and pH) from the full state vector. Rows correspond to EC and pH; VFA is not directly measurable.
`Q`	State cost matrix (LQR)	3×3 diagonal matrix penalizing state error in the LQR cost function. Diagonal entries diag([0.01, 0.01, 1000]) weight pH error 100,000× more heavily than VFA and EC error.
`R`	Input cost matrix (LQR)	2×2 diagonal matrix penalizing control input magnitude in the LQR cost function. Diagonal entries diag([0.01, 0.0001]) assign the urine pump a penalty 100× larger than the apple juice pump.
`P`	DARE solution matrix	3×3 positive semidefinite matrix solving the Discrete Algebraic Riccati Equation. Encodes the optimal infinite-horizon cost-to-go and is used to compute the LQR gain K.
`K`	LQR gain matrix	2×3 matrix mapping the state error vector ($\hat{x}$ − x_sp) to corrective pump doses. Computed as $K = (R + B_d^T P B_d)^{-1} B_d^T P A_d$.
`L`	Kalman observer gain matrix	3×2 matrix injecting the sensor innovation into the state estimate at each 5-second step. Computed as the steady-state Kalman gain by solving the DARE with Q_proc and R_meas.
`Q_proc`	Process noise covariance (Kalman)	3×3 diagonal matrix representing process noise covariance. Diagonal entries diag([0.010, 0.149, 0.001]) reflect relative confidence in the VFA, EC, and pH model predictions respectively.
`R_meas`	Measurement noise covariance (Kalman)	2×2 diagonal matrix representing sensor measurement noise variance. Diagonal entries diag([0.157, 0.001]) correspond to EC noise ≈ 0.4 mS/cm and pH noise ≈ 0.02 units (1-sigma).

Symbol	Name	Definition
x	State vector	3×1 column vector [VFA (mg/L); EC (mS/cm); pH] representing the true internal state of the reactor at a given time step.
x̂	Observer state estimate	3×1 column vector containing the Kalman filter observer's estimate of the true state x, updated at every 5-second sample using EC and pH measurements.
x_sp	Setpoint vector	3×1 column vector [50.25 mg/L; 10.0 mS/cm; 7.2] defining the desired operating point. The controller drives x̂ → x_sp.
u	Control input vector	2×1 column vector [u_urine (mL/min); u_AJ (mL/min)] representing the commanded pump doses applied at each 60-second dose update.
y	Measurement vector	2×1 column vector [EC (mS/cm); pH] containing the sensor readings available in real time, used to correct the observer estimate and update the Kalman gain.
D	Affine disturbance offset	3×1 vector [0.035; −0.164; 0.011] representing residual biological and electrochemical drift around the LQR operating point, identified from closed-loop experimental data. Set to zero in simulation to ensure the model equilibrium is exact at x_sp.
Σ	Covariance matrix (Kalman filter)	3×3 positive semidefinite matrix tracking the observer's uncertainty in the state estimate. Propagated and updated at each 5-second step via the Kalman predict-correct cycle.

Chapter 1

Introduction and Literature Review

1.1 Climate Change and Energy Consumption

Changes in the Earth's climate since the mid-twentieth century demonstrate that global average temperatures have been increasing by 0.06 degrees Celsius per decade since 1900. This sustained warming trend has led to accelerating melting of the polar ice caps, rising sea levels, and shifting of habitable zones worldwide. The Intergovernmental Panel on Climate Change has concluded with high confidence that these changes are directly attributable to human activities, particularly the combustion of fossil fuel-based energy sources. In the absence of significant mitigation efforts, global temperatures are projected to rise by 1.5 degrees Celsius above pre-industrial levels within the next two decades, crossing thresholds associated with substantially increased risk of irreversible ecological and economic disruption.

Increased energy consumption is a key driver of continued dependence on non-renewable energy sources. The primary global sectors driving this consumption are manufacturing and mining at 37%, heating and lighting of buildings at 30%, transportation at 28%, and agriculture and water management at 5%. Within the agriculture and water management category, wastewater treatment accounts for approximately 2% of total global energy use, a figure that rises to 30 to 40% of total municipal energy budgets in urban centers. The energy intensity of wastewater treatment therefore represents a meaningful

target for both efficiency improvements and for the development of technologies that convert waste streams into energy rather than consuming it[1].

Microbial fuel cells occupy a unique position in this landscape as a technology capable of simultaneously treating wastewater and generating electricity from the organic matter that conventional treatment systems consume energy to remove. This dual function positions MFCs as particularly attractive for resource-constrained settings where both energy access and wastewater treatment are limiting factors.

1.2 The Global Wastewater Crisis

Serious concerns in global public health have been exacerbated by inadequate wastewater treatment infrastructure. According to the World Health Organization and Pruss-Usten et al. (2023)[2], contaminated drinking water, improper sanitation, and poor hygiene collectively result in 1.4 million deaths annually, representing 2.5% of total global mortality in 2019. Over one million of these deaths occur each year in children under the age of five due to diarrheal diseases.

More than 90% of diarrheal disease deaths among low and middle income countries are concentrated in children aged under five, with 88% of those deaths occurring in Sub-Saharan Africa and South Asia [3] In 2022, 1.7 billion people relied on drinking water sources

[1] [1] NOAA National Centers for Environmental Information. (2024). Climate at a glance: Global surface temperature anomalies. Retrieved from https://www.ncei.noaa.gov

[2] [2] Pruss-Usten, A., Wolf, J., Corvalan, C., Bos, R., and Neira, M. (2023). Burden of disease from inadequate water, sanitation and hygiene for selected adverse health outcomes. International Journal of Hygiene and Environmental Health, 255, 114063.

[3] [11] L. Liu, S. Oza, D. Hogan, J. Perin, I. Rudan, J. E. Lawn, S. Cousens, C. Mathers, and R. E. Black, "Global, regional, and national causes of under-5 mortality in 2000-15: an updated systematic analysis with implications for the Sustainable Development Goals," The Lancet, vol. 388, no. 10063, pp. 3027-3035, 2023.

contaminated with fecal matter [4] These statistics reflect not only a humanitarian crisis but also a systemic infrastructure gap: the populations most affected by waterborne disease are precisely those with the least access to energy-intensive centralized treatment systems.

Numerous scientific disciplines are investigating the use of technologies including membrane filtration, advanced oxidation processes, constructed wetlands, and bioelectrochemical systems to treat wastewater and address this global challenge [5] Among these, bioelectrochemical systems including microbial fuel cells are of particular interest because they can function at small scale, at ambient temperature, and without external energy inputs, making them suitable candidates for decentralized treatment in resource-limited settings. The challenge is not whether MFCs can treat wastewater and generate power, which has been demonstrated repeatedly in laboratory settings, but rather how to maintain stable operation under the variable conditions encountered in real deployments. This thesis directly addresses that challenge through the development and validation of a feedback control system.

1.3 Microbial Fuel Cell Fundamentals

A microbial fuel cell (MFC) is a bio-electrochemical device that generates electricity through the bacterial metabolism of organic material. The cell consists of two chambers: an anode compartment and a cathode compartment. Both electrodes are

---

[4] [12] World Health Organization, Drinking-Water Fact Sheet. Geneva: WHO Press, 2023. [Online]. Available: https://www.who.int

[5] [13] A. Mojiri, Z. Ahmad, R. M. Tajuddin, and M. A. Kamaruddin, "A comprehensive review of hybrid constructed wetland-microbial fuel cell systems for wastewater treatment," Journal of Hazardous Materials, vol. 461, p. 132585, 2024.

constructed from non-corrosive, electrically conductive materials such as carbon veil fiber, graphite felt, or stainless-steel mesh. The anode is coated with a biofilm of electroactive bacteria surrounded by a nutrient-rich substrate. The two chambers are separated by a proton exchange membrane (PEM), a highly porous, low-resistance barrier that allows protons to pass freely while preventing bacteria and larger molecules from crossing.

In the anode compartment, electroactive bacteria such as Geobacter Sulfurreducens and Shewanella Oneidensis oxidize organic compounds in a process that releases electrons and protons. Rather than transferring these electrons to dissolved oxygen as in aerobic respiration, these exoelectrogenic organisms transfer electrons directly to the anode electrode via outer membrane cytochromes, conductive pili, or soluble electron mediators. This extracellular electron transfer is the defining feature of exoelectrogenic metabolism and the basis for MFC electricity generation[6].

In the cathode chamber, an electrode is exposed to water and oxygen under aerobic conditions. Protons generated in the anode chamber migrate through the PEM toward the cathode, driven by their electrochemical affinity for oxygen. Simultaneously, the electrons released at the anode travel through an external circuit to the cathode, where they combine with protons and oxygen to form water. This electron flow through the external circuit constitutes a useful electrical current that can be harvested to power external loads.

A central advantage of MFCs over conventional anaerobic digestion is that the MFCs capture electrons as they are released rather than allowing them to reduce carbon dioxide to methane. Methane is a greenhouse gas approximately 80 times more potent than carbon dioxide. Methane is produced *en masse* through natural anaerobic digestion and

[6] [3] Logan, B. E., et al. (2006). Microbial fuel cells: methodology and technology. Environmental Science and Technology, 40(17), 5181-5192.

animal metabolic processes worldwide. Summed over a substantial time period, its prevention is clearly an essential environmental requirement for a sustainable future. By diverting organic carbon through the electrochemical pathway rather than the methanogenic pathway, MFCs simultaneously treat wastewater, prevent methane emissions, and generate electricity, a trifecta of environmental benefits not achievable by any other single conventional technology.

1.4 Historical Development of Microbial Fuel Cells

The first documented MFC appeared in 1911 when Michael Potter reported bioelectricity generation in the *Proceedings of the Royal Society of London.* Potter achieved an open-circuit voltage of 0.32 V, comparable to voltage values reported in contemporary laboratories, though current and thus power output remained negligible. His apparatus employed a thick (approx. 10 mm) clay pot as a proton exchange membrane and platinum electrodes, with yeast incubated in a glucose-water substrate as feedstock (Potter, 1911)[7]. Although electricity production was minimal, this demonstration established proof of concept for bacterial electricity generation.

The field saw little practical progress for more than five decades following Potter's initial work[8]. Renewed interest emerged in the 1980s along with increasing social awareness of environmental impact. The technology accelerated substantially in the 1990s and 2000s as advances in molecular biology enabled the identification of the

[7] Potter, M. C. (1911). Electrical effects accompanying the decomposition of organic compounds. Proceedings of the Royal Society of London, Series B, 84(571), 260-276.

[8] [14] K. Lewis, "Symposium on bioelectrochemistry of microorganisms. IV. Biochemical fuel cells," Bacteriological Reviews, vol. 30, no. 1, pp. 101-113, 1966.

exoelectrogenic organisms responsible for electron transfer and the mechanistic understanding of how they interact with electrode surfaces. This period produced systematic examinations of electrode materials, reactor configurations, scaling effects, internal resistance, and COD removal efficiency, providing foundational evidence for the capacity of MFCs to treat wastewater by diverting organic carbon from methanogenic pathways[9].

Subsequent work by numerous research groups established the key parameters governing MFC performance including electrode material properties, membrane characteristics, biofilm development, substrate composition, and operating conditions.[10] This body of literature provides the empirical foundation upon which the modeling and control work presented in this thesis is built.

1.5 Advances in Power Density Through Miniaturization

1.5.1 Early Miniaturization Studies

Ieropoulos and colleagues achieved significant improvements in volumetric power density in 2009 by substantially reducing internal reactor volume and characteristic

[9] Logan, B. E., et al. (2006). Microbial fuel cells: methodology and technology. Environmental Science and Technology, 40(17), 5181-5192.

[10] [5] Ieropoulos, I., Greenman, J., and Melhuish, C. (2009). Improved energy output levels from small-scale microbial fuel cells. Bioelectrochemistry, 78(1), 44-50.

[11] Arduino IoT Cloud (Arduino S.r.l., Italy). Data exported as CSV for post-processing in Python 3.11 using pandas 2.0 and NumPy 1.25.

dimensions of individual MFCs, thereby increasing surface area to volume ratios and minimizing internal ohmic losses. The authors developed miniaturized MFCs with anodic volumes of approximately 6.25 mL, an order of magnitude reduction compared with conventional laboratory reactors. The compact internal geometry allowed biofilm-coated electrodes to occupy nearly the full reactor volume, reducing unused electrolyte space and improving electron harvesting efficiency. These small-scale MFCs achieved volumetric power densities of approximately 69 to 115 W/m³ in combined series-parallel stacks [11] [12].

The principle underlying these improvements, that power density scales inversely with characteristic reactor dimensions, has since been validated across numerous studies and represents a dominant design paradigm in contemporary MFC research. Reducing the distance that protons must travel between electrode surfaces reduces ionic resistance, and reducing the total electrolyte solution volume concentrates the exoelectrogenic biofilm relative to the reactor space it must support.

1.5.2 Ceramic MFC Development

Further improvements emerged in 2014 through extreme miniaturization of ceramic MFCs. Ieropoulos and colleagues developed milliliter-scale ceramic MFCs using thin ceramic cylinders with an internal liquid volume of approximately 3.88 mL. The anode was constructed from carbon veil fiber coated with activated carbon, positioned inside the

[11]

[12] I. A. Ieropoulos, J. Greenman, C. Melhuish, and I. Horsfield, "Miniature microbial fuel cells and stacks for urine use in remote locations," Physical Chemistry Chemical Physics, vol. 16, no. 30, pp. 15862-15870, 2014.

ceramic cylinder, while an air-breathing cathode attached directly to the outer ceramic surface minimized electrode separation and reduced ionic transport distances.

This compact cylindrical geometry yielded dramatically increased volumetric power density. The carbon veil plus activated carbon cathode configuration produced 0.85 mW and 219 W/m³. A graphene-modified carbon veil activated carbon cathode produced 312 W/m³. An Fe_3O_4 magnetite-modified coating produced 288 W/m³. These values represent more than an order of magnitude improvement compared with larger ceramic MFC reactors operating at liter scales[13] The ceramic membrane itself was central to this improvement: terracotta and other earthenware ceramics provide a low-cost, mechanically robust alternative to polymer PEMs, with proton conductivity adequate for MFC operation at these scales.

1.6 Long-Term Performance and Scalability

1.6.1 Extended Operation Studies

Gajda et al. (2020)[14] demonstrated the practical scalability and longevity of urine-fed MFCs through low-cost ceramic architectures. In a 19-month study, the authors evaluated ceramic MFCs operated on neat human urine using terracotta cylindrical separators and carbon veil fiber electrodes. Individual units employed short terracotta

[13] Gajda, I., Greenman, J., Melhuish, C., and Ieropoulos, I. A. (2020). Electricity and disinfectant production from wastewater: Microbial fuel cell as a self-powered electrolyser. Scientific Reports, 10, 1-11.

[14] [16] I. A. Ieropoulos, A. Stinchcombe, I. Gajda, S. Forbes, P. Ledezma, B. Zhu, J. Greenman, C. Melhuish, and J. Hanks, "Pee power urinal - microbial fuel cell technology field trials in the context of sanitation," Environmental Science: Water Research and Technology, vol. 2, no. 2, pp. 336-343, 2016.

cylinders approximately 50 mm in height and 14 mm in diameter, with 3 mm thick ceramic walls functioning as proton exchange membranes. Individual units achieved peak power outputs of 1.56 mW with a volumetric power density of 22.3 W/m³, exhibiting only a 20% performance loss after 350 days. When scaled to 66 units in cascade configuration, systems reached 13.9 W/m³ while enabling greater than 92% cumulative COD removal.

1.6.2 Operational Challenges

Despite demonstrated durability, several persistent long-term operational challenges have been identified as critical for practical deployment. Gradual power losses are attributed to biofouling and inorganic salt deposition on separator and cathode surfaces, increasing internal resistance and limiting ion transport. Feedstock depletion, variable urine composition, and intermittent starvation periods introduce temporal power fluctuations and can alter the electroactive microbial community.

In field-scale deployments, Ieropoulos and colleagues (2016)[15] reported that excessively frequent feeding acted as a functional shock to the system. High-frequency feed pulses led to power output decline compared with more moderate feeding regimes, attributed to disruption of electrochemical stratification and stress or washout of the electroactive biofilm. This finding demonstrates that the timing and magnitude of feedstock delivery critically affects long-term MFC performance, providing direct motivation for the

[15] [17] G. C. Gil, I. S. Chang, B. H. Kim, M. Kim, J. K. Jang, H. S. Park, and H. J. Kim, "Operational parameters affecting the performance of a mediator-less microbial fuel cell," Biosensors and Bioelectronics, vol. 18, no. 4, pp. 327-334, 2003.

development of feedback-controlled dosing strategies that can adapt delivery rates to the biological state of the reactor.

1.7 Key Biochemical Parameters Affecting MFC Performance

1.7.1 Effects on Acidity/Alkalinity (pH)

Acidity/alkalinity (pH) directly influences microbial metabolism in the anode region and electrochemical reaction rates at the cathode. Gil et al. (2003)[16] identified the problem of proton buildup, showing that anode pH values in unbuffered systems can decrease below 5.0, leading to greater than 70% current reduction due to inhibitory effects on electroactive microorganisms. At pH values below 6.0, the activity of cytochrome-based outer membrane electron transfer proteins is substantially reduced, and the metabolic pathways supporting acetate oxidation shift away from the exoelectrogenic route toward less productive fermentation pathways.

Gil et al. (2008)[17] identified a performance optimum at anode chamber pH of 7.0, with a 50% power decrease at pH 6.0 and an over 80% power decrease at pH 5.0. Conversely, more acidic catholytes in the range of pH 2 to 4 increased cathode reaction rates but led to accelerated material degradation, creating a design tension between anodic and cathodic pH optimization.

[16] [18] Z. He, Y. Huang, A. K. Manohar, and F. Mansfeld, "Effect of electrolyte pH on the rate of the anodic and cathodic reactions in an air-cathode microbial fuel cell," Bioelectrochemistry, vol. 74, no. 1, pp. 78-82, 2008.

[17] [19] R. Kumar, L. Singh, Z. A. Wahid, and M. F. M. Din, "Exoelectrogens: Recent advances in molecular drivers involved in extracellular electron transfer and strategies used to improve it for microbial fuel cell applications," Renewable and Sustainable Energy Reviews, vol. 56, pp. 1322-1336, 2021.

Kumar et al. (2021)[18] conducted system-level analysis of pH imbalance costs. Extreme transmembrane pH gradients (exceeding 3.0 units) decreased power density from 40% to 60% more than those compared with low pH differentials of 1.5 units or less. This reduction was attributed to increased internal resistance and shifts in anodic microbial populations from efficient electrogens such as Geobacter to lower-productivity fermentative species. Slightly alkaline anolyte pH in the range of 7.5 to 8.0 increased Coulombic efficiency by 15 to 25% through reduced methanogenesis. These findings establish pH regulation as the highest-priority biochemical control objective in a multi-variable MFC control system, and this priority is directly reflected in the controller design presented in this thesis by assigning a state cost penalty on pH error orders of magnitude larger than those on VFA and EC error.

1.7.2 Salinity and Electrical Conductivity Effects

Salinity affects both microbial stress responses and electrochemical properties of the MFC. Lefebvre et al. (2012) demonstrated the acute inhibitory effect of osmotic shock: a sudden increase of 20 g/L NaCl in an acetate-fed MFC resulted in a 65% current density decrease, primarily from microbial plasmolysis caused by rapid water loss from the bacterial cytoplasm.

[18] Lefebvre, O., et al. (2012). Effect of salinity on microbial fuel cell performance. Bioresource Technology, 112, 336-341.

Adaptation studies demonstrate the capability of halotolerant microbial consortia to maintain high performance under elevated salinity. Larrosa et al. (2010)[19] found that MFCs fed marine sediments at 35 g/L NaCl produced stable power of approximately 120 mW/m², significantly higher than freshwater-fed controls, selecting for halotolerant exoelectrogenic species such as Desulfuromonas. The higher conductivity of saline environments directly reduces solution resistance and therefore also reduces ohmic losses in the electrochemical circuit.

Ping et al. (2015)[20] clarified the electrochemical trade-offs involved: higher ionic strength not only reduces solution resistance through increased charge carrier concentration but also can simultaneously increase charge transfer resistance at the electrode-biofilm interface through competitive ion adsorption and biofilm structural effects. This non-monotonic relationship between ionic strength and performance means that electrical conductivity must be maintained within an optimal range rather than simply maximized, providing the motivation for EC as a state variable in the control system developed in this thesis.

1.7.3 Volatile Fatty Acids

Volatile fatty acids (VFAs) serve as the primary substrate for electroactive bacteria in the anode biofilm. Acetate is the most commonly studied VFA in MFC research because it is the direct substrate for Geobacter sulfurreducens, the most well-characterized

[19] [20] Q. Ping, B. Cohen, C. Dosoretz, and Z. He, "Long-term investigation of fouling of cation and anion exchange membranes in microbial desalination cells," Desalination, vol. 357, pp. 64-72, 2015.

[20] [21] J. Jiang, Y. Zhang, K. Li, Q. Wang, C. Gong, and M. Li, "Volatile fatty acid production from organic wastes by anaerobic fermentation," Bioresource Technology, vol. 137, pp. 341-347, 2013.

exoelectrogenic organism. However, real waste streams produce a complex mixture of VFAs through the fermentation of more complex organic molecules, and the speciation of this mixture depends on feedstock composition, hydraulic retention time, temperature, and microbial community structure.

Jiang and colleagues (2013)[21] characterized VFA production from organic wastes by anaerobic fermentation, establishing first-order kinetic parameters relevant to mixed-feedstock systems. Dionisi and colleagues (2019)[22] demonstrated that VFA yield and speciation depend on hydraulic retention time and organic loading rate. Elevated VFA concentrations above a critical threshold can inhibit methanogenic and exoelectrogenic communities through acidification, because VFA dissociation releases protons that suppress pH and shift the microbial community toward acid-tolerant fermenters rather than exoelectrogens. Hajarnis and Ranade (1994)[23] established inhibitory thresholds for individual VFAs, with inhibition of methanogenic activity observable at concentrations as low as 150 mg/L for propionic acid. Active VFA management is therefore essential for sustained high-performance MFC operation.

1.7.4 Thermal Effects

---

[21] [22] D. Dionisi, J. A. Anderson, F. Aulenta, A. McCue, and G. Paton, "Production of volatile fatty acids from organic wastes by anaerobic fermentation," Waste and Biomass Valorization, vol. 11, no. 5, pp. 1609-1625, 2019.

[22] S. R. Hajarnis and D. R. Ranade, "Inhibition of methanogens by n- and iso-volatile fatty acids," World Journal of Microbiology and Biotechnology, vol. 10, no. 3, pp. 350-351, 1994.

[23] [8] Burch, G., Chatterjee, S., and Mansour, M. N. (2022). Mathematical modeling and feedback control in microbial fuel cell systems: A review. Bioresource Technology, 344, 126307.

Temperature influences both microbial metabolic rates and electrochemical reaction kinetics. Most exoelectrogenic bacteria are mesophilic, with optimal activity in the range of 30 to 37 degrees Celsius. At temperatures below 20 degrees Celsius, metabolic rates decline substantially and biofilm stability can be affected. At temperatures above 40 degrees Celsius, competing thermophilic organisms may outcompete mesophilic exoelectrogens for substrate, reducing Coulombic efficiency.

In the experiments reported in this thesis, reactor temperature was monitored continuously using a DS18B20 digital temperature probe but was not actively controlled. The experimental system was maintained at ambient laboratory temperature, which fluctuated around 22 °C 5 °C throughout the experimental period. This temperature range is within the viable operating range for the exoelectrogenic community in the reactor and was not identified as a significant source of performance variation between LQR and baseline phases, as both phases experienced identical ambient temperature conditions.

1.8 Microbial Fuel Cell Control Systems

The complex, time-variant, nonlinear character of MFC systems poses significant modeling and control challenges. Most experimental studies have focused on characterization rather than feedback control[24]. The nonlinearity of microbial metabolism, the slow time constants of biofilm adaptation, and the difficulty of measuring internal biochemical states without disrupting the system have all contributed to the relative scarcity of control systems work in this field compared with the extensive experimental characterization literature.

[24] R. Patel and D. Deb, "Parametrized control-oriented mathematical model and adaptive backstepping control of a single chamber single population microbial fuel cell," Journal of Power Sources, vol. 396, pp. 599-605, 2018.

Various control strategies have been proposed. Adaptive fuzzy control schemes have been demonstrated for setpoint tracking under dynamic loads, achieving settling time improvements compared with fixed-parameter PID controllers. Machine learning-based approaches have demonstrated 65% settling time improvements over linear controllers in single-input systems. Model predictive control has been applied to wastewater treatment more broadly, with the ability to enforce input constraints explicitly. However, all of these approaches have been demonstrated primarily in single-input systems or simulations, with limited experimental validation in multi-input configurations.

Mathematical modeling based on mass-charge balance principles can support advanced nonlinear controller development.[25] Such models have been used to design backstepping controllers for single-population, single-chamber MFCs. However, the extension of these approaches to multi-input systems with unmeasured states and real-time hardware implementation has not been previously demonstrated in the experimental literature.

Upscaled MFC implementations such as the Pee Power and Urintricity systems developed by Ieropoulos and colleagues demonstrated practical field applications achieving continuous power production for off-grid electricity generation. However, these scaled systems utilized urine as a single feedstock and did not incorporate real-time feedback control for multiple input substrates, relying instead on passive hydraulic design and fixed-rate feeding.

[25] S. H. A. Hassan, S. M. F. G. El-Rab, M. Rahimnejad, M. Ghasemi, J. H. Joo, Y. Sik-Ok, I. S. Kim, and S. E. Oh, "Electricity generation from sugarcane molasses using a microbial fuel cell with a novel sandwiched membrane," Journal of Industrial and Engineering Chemistry, vol. 73, pp. 249-256, 2019.

1.8.1 Multi-Input Substrate Opportunities

The incorporation of multiple input substrates presents significant opportunities for power enhancement and operational flexibility. Iron oxide (Fe3O4) addition to the substrate substantially improves MFC performance, yielding an 86% power increase compared with bare carbon felt electrodes (166 plus or minus 11 mW/m²). Hassan and colleagues (2019)[26] reported that carbohydrate-based substrates such as sugarcane molasses produced maximum power densities up to 188.5 mW/m² with an open-circuit voltage of 990 mV and over 81.7% COD removal, compared with typical urine-fed power densities of 23 to 70 mW/m².

The combination of a carbohydrate-based feedstock with a urine-based feedstock is particularly attractive from a control perspective because the two substrates have opposing effects on pH: carbohydrate fermentation produces VFAs and is acidifying, while fermented urine is alkaline due to ammonia content. Delivering these two substrates as separately controlled inputs creates a natural pH actuation mechanism, where the ratio of urine to apple juice determines whether the reactor pH rises or falls. This is precisely the control architecture exploited in the LQR designed in this thesis.

1.9 Research Gap and Objectives

Despite recent progress in MFC technology (power densities, material design, electrode architecture, et al.) a critical gap exists in the development of control systems for multi-input microbial fuel cells. Although the effects of pH, electrical conductivity, and substrate concentration on MFC performance have been investigated individually, traditional control models focus on single-input systems or employ simplified models that

[26] See Appendix A.

fail to capture coupled behavior among multiple state variables. To the author's knowledge, no published work has demonstrated physics-based dynamic state-space models used for closed-loop feedback control of a multi-input biological electrochemical system with experimental hardware validation. There exists a need to integrate multi-substrate feeding systems, coupled-state variable dynamics, real-time observer estimation of unmeasured states, and full-state feedback control validated on experimental hardware.

Furthermore, while advantageous multi-feedstock approaches have been demonstrated individually, no studies were found that addressed the implementation of control systems that both regulate performance fluctuations for long-term stability and simultaneously optimize the synergistic benefits of multiple feedstocks. This research gap constrains practical MFC deployment under variable feedstock conditions and environmental perturbations. The present thesis directly addresses this need.

This thesis addresses the research gap by developing a physics-based dynamic model for feedback control of a multi-input microbial fuel cell system. The research designs, implements, and experimentally validates a control architecture regulating three state variables: volatile fatty acid (VFA) concentration, electrical conductivity (EC), and pH in an 875 mL continuously stirred tank reactor fed by fermented urine and apple juice as manipulated inputs.

The principal contributions are: (1) a physics-based discrete-time state-space model of coupled VFA, EC, and pH dynamics identified from experimental data using Differential Evolution and Nelder-Mead optimization; (2) a discrete-time Kalman filter observer for estimation of the unmeasured VFA state from EC and pH measurements, implemented in real-time on an Arduino GIGA R1 WiFi microcontroller; (3) a discrete-time LQR for multi-input feedback control with a cost matrix design that reflects the differential biological importance of pH, EC, and VFA regulation; (4) experimental comparison of LQR adaptive dosing against a matched open-loop baseline across three experimental campaigns with performance quantified by integrated MFC voltage and ORP; and (5) the complete firmware implementation[27] enabling replication of this work.

[27]

Chapter 2

System Design

2.1 Manufacturing Methods

A terracotta-based clay body sourced from Portland Pottery, MA was conditioned through manual kneading and hydration to achieve a uniform, plastically workable consistency. The prepared clay was loaded into a caulking gun-type clay extruder fitted with a custom annular die fabricated from 6061-T651 aluminum plate machined via waterjet cutting. The die consisted of a coaxial outer plate and inner mandrel, with the annular gap defining a tube wall thickness of approximately 0.5 mm. During extrusion, a combined compressive force from the plunger and a gentle tensile pulling force on the emerging tube were applied simultaneously to ensure dimensional uniformity and minimize internal stresses. The extruded tubes were cut to approximately 80 mm in length immediately after formation using a scalpel blade.

Following extrusion, the green tubes were dried using a rotational drying method to prevent gravitational deformation. Each tube was mounted on a cylindrical wooden mandrel of 6 mm outer diameter and rotated continuously for approximately 10 hours using a motorized roller apparatus (commercial hot dog roller machine). This rotation maintained the circular cross-section of the tube during drying by distributing the gravitational load uniformly around the circumference rather than allowing the tube to sag on one side.

After drying, the tubes were fired in Vulcan kilns located in the Boston University EPIC facility using a multi-stage thermal profile: dehydration at approximately 300 degrees Celsius for 3 hours, sintering at approximately 1000 degrees Celsius for 3 hours, and controlled cooling at approximately 200 degrees Celsius for 3 hours, followed by gradual return to ambient temperature. This firing schedule corresponds to earthenware cone 06 and is standard for terracotta clay bodies. The resulting ceramic membranes exhibited consistent wall thickness, maintained circular cross-section, and demonstrated sufficient mechanical robustness to withstand handling and insertion into the reactor assembly. Porosity was confirmed by the passage of water through the walls under gentle pressure, consistent with the proton exchange requirements of MFC operation.

The ceramic membrane approach was selected over commercial polymer PEMs based on the precedent established by Ieropoulos and colleagues (2014) and the substantially lower cost of terracotta clay relative to Nafion and similar fluoropolymer membranes. The ability to fabricate membranes with custom geometry in a university setting without specialized equipment is a key practical advantage for future scale-up and replication.

[INSERT Figure 2.1: Photograph of fired ceramic membrane tubes showing consistent wall thickness and circular cross-section. Scale bar = 10 mm. Publication-quality image required — no screen captures.]

2.2 MFC Assembly

A microbial fuel cell was assembled and inserted into the reactor to enable bioelectrochemical energy harvesting from the anaerobic digestion process. The anode was constructed from a 76 mm × 203 mm sheet of carbon veil fiber wrapped around a ceramic

separator tube, which served as the ion exchange membrane between the anode and cathode compartments. The carbon veil fiber was selected for its high surface area, flexibility, and compatibility with biofilm formation. The veil was secured using 0.5 mm outer diameter grade 304 stainless steel wire, which simultaneously served as the current collector for the anode, providing a low-resistance electrical pathway from the biofilm to the external circuit. One end of the assembly was sealed with silicone to prevent direct fluid communication between the anode and cathode compartments at that end.

The cathode was constructed from a 76 mm × 38 mm sheet of carbon veil fiber affixed to a METALLIXITY compression spring (1.2 mm outer diameter, 305 mm free length, grade 304 stainless steel) using hot glue, such that the carbon veil fiber was uniformly wrapped around the spring structure. This cathode assembly was inserted into the interior of the ceramic separator, with the spring providing structural support and maintaining consistent contact pressure between the carbon veil and the inner surface of the ceramic wall.

To supply the cathode with a continuous source of dissolved oxygen, a 3 mm outer diameter aquarium tube was threaded through the interior of the spring and connected to an AquaMiracle all-in-one aquarium air pump, delivering a constant stream of air bubbles directly to the cathode surface throughout all experimental runs. This configuration maintained aerobic conditions at the cathode while the surrounding reactor bulk remained anaerobic, essential for the proton-oxygen recombination reaction that closes the electrochemical circuit.

[INSERT Figure 2.2: Schematic of MFC assembly showing anode carbon veil wrapped around ceramic tube, cathode inside with spring support, air delivery tube, and current collector wire connections. Publication-quality image required — no screen captures.]

2.3 Reactor Construction

The 875 mL anaerobic digester reactor body was fabricated from 3 mm thick clear acrylic sheet, chosen for its chemical resistance to the reactor contents, optical transparency allowing visual inspection of reactor conditions without opening the system, and structural rigidity at the required scale. All panels were designed in Onshape CAD software and laser cut to their final dimensions using a CO2 laser cutter. The reactor interior measures 130 mm x 130 mm in plan, with an assembled reactor body height of 66.4 mm and an overall height of 122.2 mm including the integrated tray stand.

The six panels comprising the bottom, top lid, two side walls, one input side panel, and one output side panel were joined using Gorilla Super Glue in a finger-joint configuration that provided mechanical interlocking at all panel edges without the need for external fasteners. The finger-joint geometry was designed in Onshape and included in the laser cut files, ensuring consistent alignment during assembly. The top lid was left unglued to allow removal for maintenance, inoculation, and sampling without disrupting the sealed reactor body below.

All interior seams and corner joints were then sealed with GE All Purpose 100% Silicone Caulk and allowed to cure at room temperature for one week prior to filling with liquid. The silicone was applied in a continuous bead along all interior seams and smoothed to eliminate voids. Water tightness was verified by filling the reactor with tap water and observing for leaks over a 24-hour period before inoculation.

Four sensor ports were laser cut into the top lid: three at 12 mm diameter for the pH, ORP, and EC probes, and one at 6 mm diameter for the temperature probe. Sensors were secured using custom laser-cut acrylic holders with a 12.5 mm outer diameter and 10 mm inner bore, friction-fit into the lid ports and holding each probe at a consistent immersion depth of approximately 40 mm below the liquid surface. A 12.5 mm port accommodated the overhead continuous stirrer shaft for mechanical agitation, maintaining a well-mixed anaerobic environment throughout each experimental run.

Three 6.17 mm diameter input ports were laser cut into one side panel to accommodate the peristaltic pump inlet tubing delivering urine, apple juice, and phosphoric acid into the reactor interior. All three inputs used 3 mm outer diameter neoprene tubing, press-fit through the ports. Of these three inputs, only urine and apple juice are manipulated by the LQR control law; phosphoric acid is delivered exclusively during the acid perturbation phases of the experimental protocol and is not part of the state-space model or controller design. A single 20.3 mm diameter output port on the opposite side panel was fitted with flexible silicone hose for effluent drainage. The MFC assembly was inserted through a dedicated 25 mm port in the top lid.

[INSERT Figure 2.3: Photograph of assembled reactor showing sensor ports in lid, input tubing connections, output port, and overhead stirrer. Scale bar = 50 mm. Publication-quality image required — no screen captures.]

2.4 Instrumentation

Continuous real-time monitoring of reactor conditions was performed using four electrochemical sensors and one temperature sensor, all interfaced with an Arduino GIGA R1 WiFi microcontroller. The Arduino GIGA R1 was selected for its dual-core ARM Cortex-M7 and Cortex-M4 processors, 16-bit ADC resolution (configured here at 12-bit for noise reduction), built-in WiFi for Arduino IoT Cloud connectivity, and sufficient digital and analog GPIO for the required sensor and pump interface.

Oxidation-reduction potential was measured using a DANOPLUS ORP electrode with a replaceable BNC socket probe connected to analog pin A1. Electrical conductivity was measured using a DFRobot Gravity Lab Grade Analog EC sensor (SKU: DFR0300, range 0 to 20 mS/cm) on analog pin A2. pH was measured using a Teyleten Robot analog pH sensor module (range pH 0 to 14) on analog pin A11. Temperature was measured using a WWZMDiB DS18B20 waterproof digital temperature probe on digital pin D10 via the OneWire protocol. MFC terminal voltage was measured by connecting the MFC output terminals directly to analog pin A3 through a voltage divider, with readings taken during the gate-on half of the relay cycle.

All sensor data were transmitted to the Arduino IoT Cloud platform for live monitoring and historical storage at 5-second intervals throughout the experiment. A relay gate on digital pin D3 alternated between ORP measurement and MFC voltage measurement on a 10-second cycle, with the first 5 seconds allocated to ORP (relay off, no current flowing) and the second 5 seconds allocated to MFC voltage (relay on, current flowing through external load). This alternating gate design prevented the ORP reading from being corrupted by the MFC current and prevented the MFC from being short-circuited during ORP measurement.

Three peristaltic pumps delivered the three input streams. The urine pump was connected to digital pin D8, the apple juice pump to D4, and the phosphoric acid pump to D2. Pump flow rates were characterized prior to experimentation by timing the delivery of measured volumes: the urine pump delivered 1.855 mL/s, the apple juice pump 1.055 mL/s, and the phosphoric acid pump 0.725 mL/s. These rates were used to compute pump actuation times from desired dose volumes in the firmware.

Chapter 3

Methodology

3.1 Experimental Setup

The experimental system consisted of an 875 mL continuously stirred tank reactor (CSTR) inoculated with anaerobic sludge obtained from a local wastewater treatment facility and connected to a single-chamber microbial fuel cell as described in Chapter 2. The reactor was operated in fed-batch mode with three peristaltic pumps delivering fermented urine, apple juice, and phosphoric acid according to the control law and phase logic described in the following sections. The MFC was operated continuously, with its output connected to the relay gate circuit that alternated between ORP measurement and voltage measurement on a 10-second cycle.

The reactor was inoculated with 100 mL of anaerobic sludge diluted into 775 mL of tap water, providing an initial microbial community capable of both VFA production through fermentation and exoelectrogenic activity through the biofilm that formed on the carbon veil anode during the initial acclimation period. A minimum of 72 hours of continuous operation on the urine and apple juice substrate mixture was allowed before the experimental protocol was initiated, to allow the biofilm to establish and reach a pseudo-steady electrochemical output.

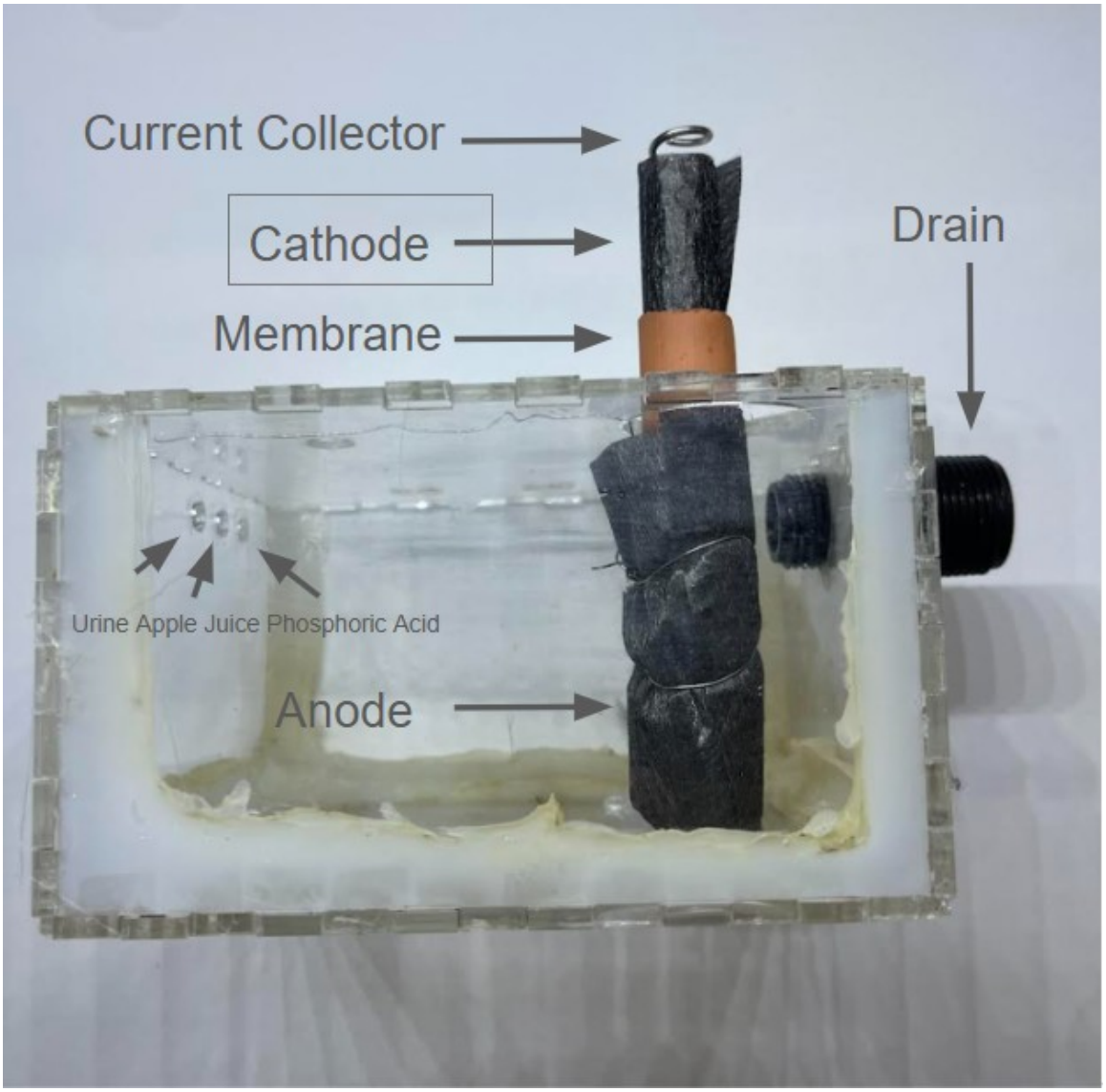

Figure 2.2. Assembled microbial fuel cell reactor (side view) showing the anode chamber, terracotta membrane separator, carbon-based cathode, current collector, and fluid drain. Electrolyte inputs (urine, apple juice, and phosphoric acid) are indicated.

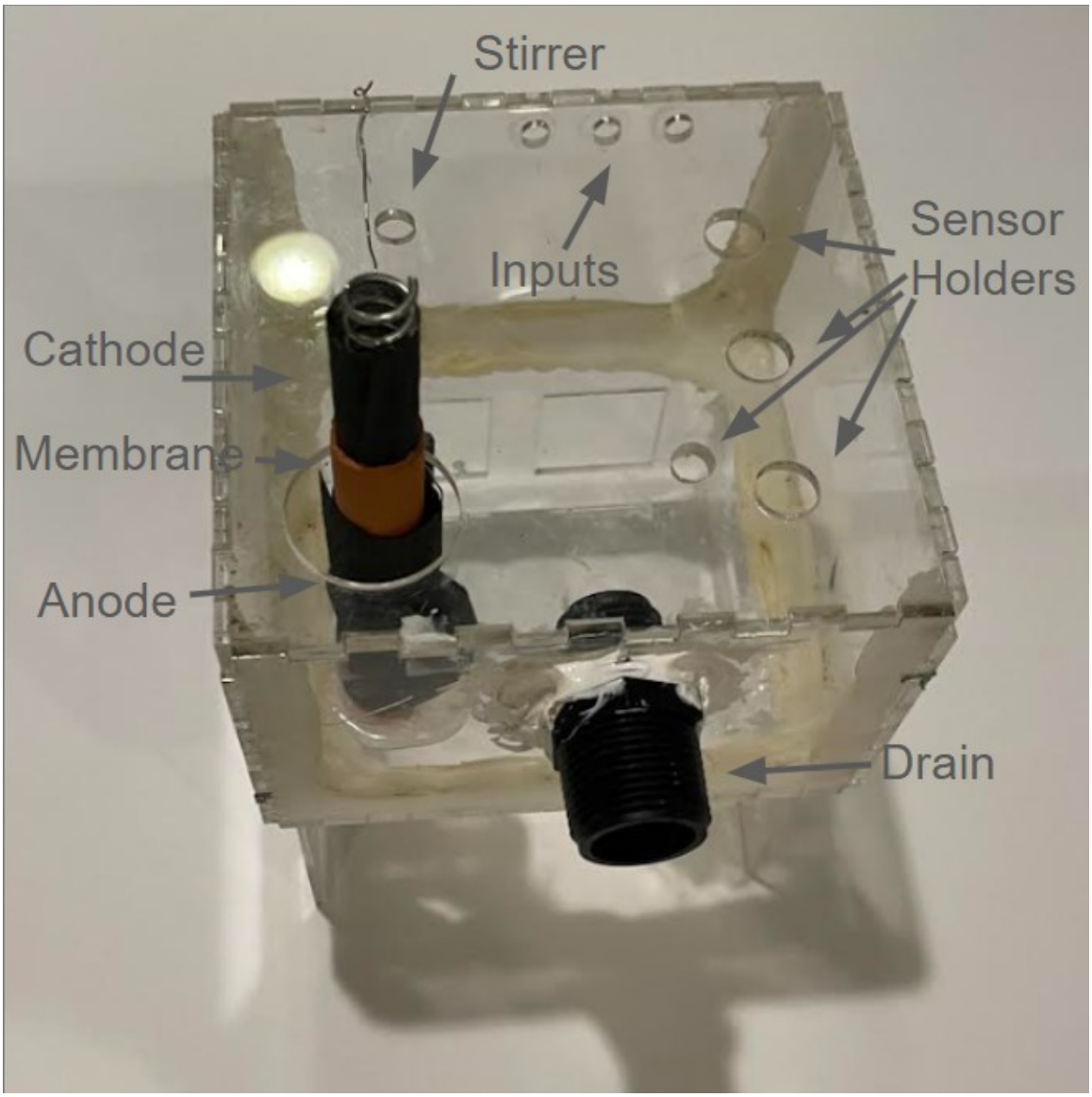

Figure 2.3. Assembled microbial fuel cell reactor (top view) showing the cathode assembly, terracotta membrane separator, anode chamber, influent inputs, integrated stirrer region, sensor holders, and effluent drain.

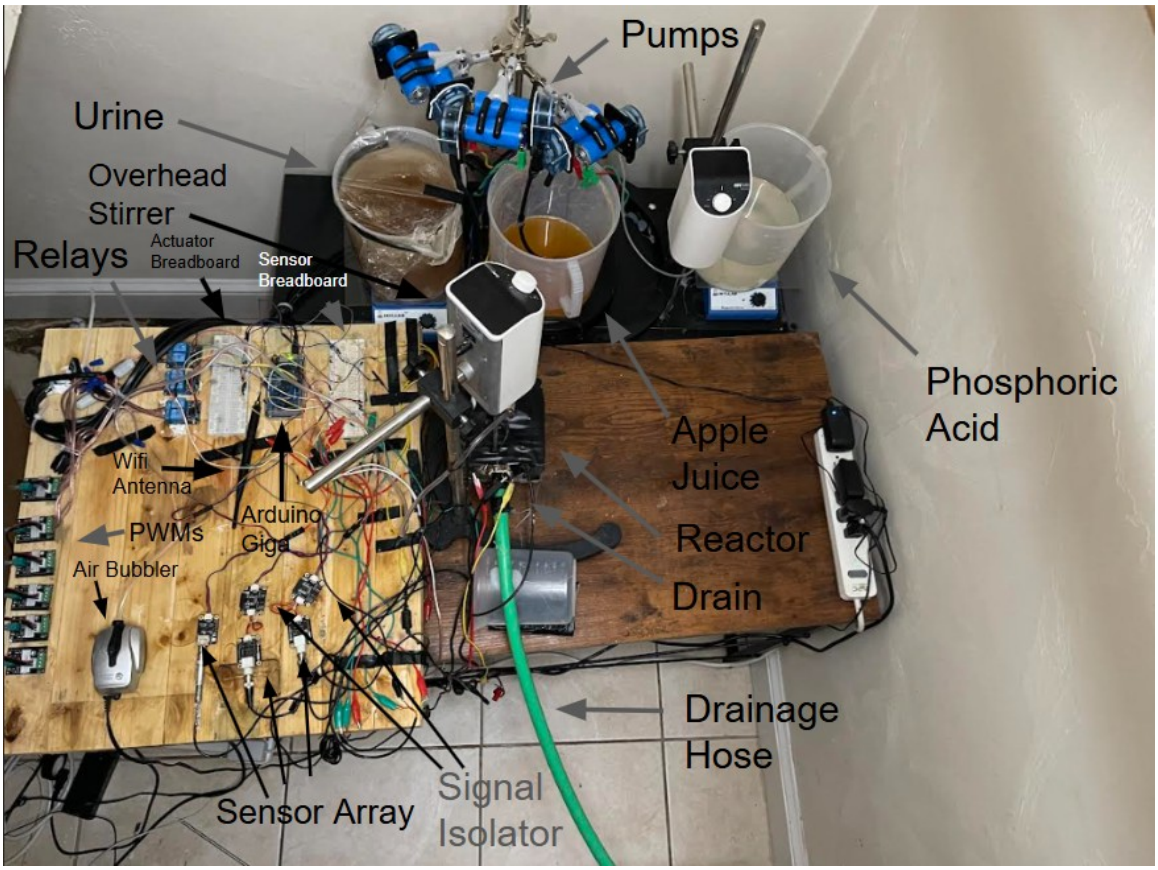

Figure 3.3. Full experimental setup of the microbial fuel cell system, including fluid reservoirs (urine, apple juice, and phosphoric acid), peristaltic pumps, reactor unit, drainage system, sensor array, control electronics (Arduino-based), and supporting circuitry.

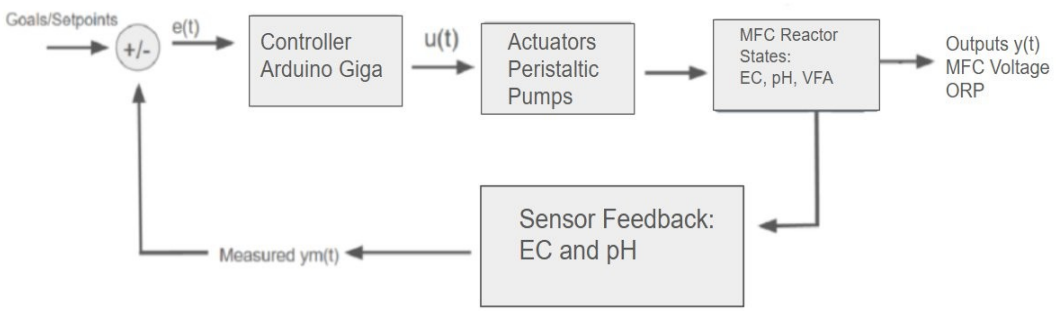

Figure 3.1. Control system block diagram. The Arduino GIGA R1 controller computes u(t) from state error e(t) measured by the Kalman filter observer and drives peristaltic pump actuators. EC and pH sensor feedback closes the loop. VFA is estimated by the observer from EC and pH measurements.

3.2 Sensors and Calibration

The pH sensor was calibrated on 28 March 2026 using a two-point linear model with pH 4.0 and pH 7.0 buffer solutions. Raw ADC voltages extracted from the Arduino serial log yielded slope PH_M = -8.2123 and intercept PH_B = 17.0046. A voltage validity check was implemented in firmware: readings with raw voltage above 3.28 V or below 0.1 V, or resulting pH values outside the range 0 to 14, were flagged as invalid and the last valid reading was retained. This prevented sensor noise spikes or disconnection events from corrupting the control calculation.

The EC sensor was recalibrated on 28 March 2026 using two conductivity standards (1.413 mS/cm and 12.88 mS/cm). A two-point linear calibration yielded a corrected slope of 13.6887 mS/cm per volt and offset of -7.2574 mS/cm, replacing the prior calibration (slope = 9.006, offset = -5.567). All EC data collected prior to recalibration were corrected

post-hoc by inverting the old calibration to recover raw voltages and reapplying the new calibration, ensuring consistent units across the full dataset used for system identification.

The ORP sensor was calibrated on 28 March 2026 using a 470 mV standard solution, yielding an offset of -92.5 mV applied as a fixed correction to the raw reading. ORP was computed from the raw ADC voltage using the formula ORP = 1320.0 - V_raw * 1000.0 - ORP_OFFSET, where V_raw is in volts.

Table 3.1. Sensor calibration parameters.

Sensor	Cal. Date	Parameter 1	Parameter 2	Method
pH	28 Mar 2026	Slope: -8.2123	Intercept: 17.0046	Two-point linear (pH 4.0, 7.0)
EC	28 Mar 2026	Slope: 13.6887 mS/V	Offset: -7.2574 mS	Two-point (1.413, 12.88 mS/cm)
ORP	28 Mar 2026	Offset: -92.5 mV	N/A	Single-point 470 mV standard
Temp	Factory	N/A	N/A	DS18B20 factory calibrated

3.3 State-Space Model

The digester was modeled as a discrete-time linear state-space system with a 5-second sample period. The fundamental modeling assumption is that the reactor is sufficiently well-mixed by the overhead stirrer that spatial gradients in pH, EC, and VFA concentration are negligible, reducing the system to a lumped parameter model. Under this assumption, the dynamics of all three state variables can be described by ordinary

differential equations, which are discretized for implementation on the digital control hardware.

The model equations are:

where the state vector

T

contains volatile fatty acid concentration in mg/L, electrical conductivity in mS/cm, and pH. The input vector

T

contains peristaltic pump flow rates in mL/min. The output vector

T

contains the two directly measurable states. VFA is an unmeasured hidden state estimated by the Kalman filter observer described in Section 3.7. The observation matrix H selects the two measured states from the full state vector.

The model is formulated in deviation variable form around the operating setpoint

TT

This formulation has the advantage of placing the equilibrium of the controlled system at the origin of the deviation variable space, simplifying the LQR cost function and enabling direct interpretation of the controller gain matrix K in terms of deviation from setpoint rather than absolute state values.

The affine offset term D_OFFSET is a constant 3×1 vector that captures steady biological disturbances including MFC proton consumption, residual microbial alkalinity production from urine fermentation not captured by the B matrix, and other constant drift terms. It was computed analytically to ensure that the model predicts zero state change at the setpoint under the baseline input

Note that D_OFFSET was identified at the prior setpoints EC0 = 8.0 mS/cm and pH0 = 7.0, and the EC term differs from the analytically correct value at EC0 = 10.0, pH0 = 7.2 by approximately 0.026 mS/cm per step. The Kalman filter observer corrects for this residual mismatch at every 5-second step via the EC innovation term.

3.4 A Matrix Identification

The A matrix describes the autonomous dynamics of the system in the absence of inputs, encoding how each state variable influences the others over a 5-second sample period. The diagonal elements of A represent self-decay rates for each state variable, while off-diagonal elements represent cross-coupling. For a biologically stable reactor, all eigenvalues of A must lie strictly inside the unit circle in the discrete-time domain, corresponding to the physical requirement that all state variables return to equilibrium after a perturbation if left without inputs.

The A matrix was identified from a 48-hour dataset collected 15 to 17 February 2026, during which the tank was operated at its natural biological equilibrium (EC

approximately 12 mS/cm, pH approximately 8.3) without active control. A grid search over candidate training windows within this dataset was performed, with Nelder-Mead optimization (scipy.optimize.minimize, Python 3.11) applied to each window to minimize the sum of squared one-step-ahead prediction errors on EC and pH simultaneously. The best window was selected by validation R-squared on a held-out portion of the data not used in identification.

A confirmatory zero-input experiment was conducted 27 to 28 March 2026 using fresh sludge, with the MFC running but all three pumps turned off for 16.3 hours. This provided a clean zero-input dataset for validating the A matrix diagonal terms. The identified pH diagonal term (0.99797) and EC diagonal term (0.98163) agreed with the February identification values to within 0.3%, confirming that the A matrix structure is robust across operating points separated by weeks and at different absolute EC and pH values.

Table 3.2. Continuous-time A matrix (A_c, units per second). Off-diagonal coupling terms confirm that EC and pH dynamics are coupled through the shared biology of the reactor.

A_c

	VFA	EC	pH
VFA	-1.389e-04	-4.167e-06	-6.944e-06
EC	+4.710e-04	-3.140e-03	+4.951e-03
pH	-3.144e-05	+2.250e-04	-3.930e-04

Table 3.3. Discrete-time A matrix (A_d, 5-second sample period). Computed from A_c via zero-order hold discretization.

A_d

	VFA	EC	pH
VFA	0.99931	-2.083e-05	-3.472e-05
EC	+2.355e-03	0.98417	-2.476e-02

pH	-1.572e-04	+1.125e-03	0.99803

The open-loop eigenvalues of A_d are:

Eigenvalue	z value	|z|	Time Constant	Stable?
z1	0.99983	0.99983	~8.0 hours	Yes
z2	0.99930	0.99930	~2.0 hours	Yes
z3	0.98266	0.98266	~4.8 minutes	Yes

All eigenvalues lie strictly inside the unit circle, confirming open-loop stability. The three distinct time constants reflect the multi-scale nature of the reactor dynamics: the fastest mode (4.8 min) corresponds to rapid electrochemical equilibration, the intermediate mode (2.0 h) to acid-base buffering capacity, and the slowest mode (8.0 h) to the overall hydraulic and biological state of the reactor.

Table 3.4. Open-loop eigenvalues of A_d. All strictly inside unit circle confirming open-loop stability.

3.5 B Matrix Identification

The B matrix maps each input flow rate in mL/min to its effect on each state variable per 5-second sample step. Each entry was identified from dedicated step-dose experiments conducted on 26 March 2026, with the exception of the VFA rows which were derived from fermentation kinetics literature. The step-dose methodology involved applying a known dose of one input while holding the other constant, then fitting the resulting sensor response to extract the individual B matrix entries.

The b_uri_pH entry was identified using Differential Evolution (scipy.optimize.differential_evolution, Python 3.11) to initialize a global search, followed by Nelder-Mead refinement (scipy.optimize.minimize), on a dataset collected during a

predominantly urine-dosed LQR run (16:07 to 16:22 EST), selected because it provided a clean window of sustained urine dosing without confounding phosphoric acid events. Differential Evolution was used to initialize the search over a broad parameter range, followed by Nelder-Mead refinement, yielding b_uri_pH = +2.491e-2, RMSE = 0.043, and R-squared = 0.796. The b_AJ_pH entry was identified from a dedicated 10-minute apple juice step experiment (22:47 to 23:05 EST) in which apple juice was dosed at 12 mL/min, yielding b_AJ_pH = -1.663e-3. This value corrected a 2.68-fold overestimation in the prior model version (v20) where b_AJ_pH had been set to -4.458e-3 based on a misidentification.

The EC entries b_uri_EC and b_AJ_EC were identified from the same experimental windows using linear regression on the EC response during the respective step doses, yielding b_uri_EC = +6.840e-4 and b_AJ_EC = -4.733e-3. These signs are physically consistent: urine has high ionic strength and raises EC, while apple juice is dilute and lowers EC. The VFA entry for apple juice (b_AJ_VFA = +1.807e-3) was taken from fermentation kinetics literature [28][29] The urine VFA entry was set to zero because stored fermented urine has a VFA concentration below 100 mg/L [30] giving an effective b_uri_VFA approximately 16 times smaller than b_AJ_VFA and therefore negligible for control purposes.

28

29

[30] The asymmetric R = diag([0.01, 0.0001]) produces a near-deadbeat third closed-loop eigenvalue $|z_3|$ = 0.011 ($\tau \approx 1.1$ s). This was identified post-experiment. A future redesign should use R = diag([1.0, 0.01]) to raise $|z_3|$ to ≈ 0.40 ($\tau \approx 5.5$ s), reducing controller aggressiveness without sacrificing pH tracking. The rate limiter in firmware v21/v22 (MAX_DOSE_RAMP = 0.5 mL/min per 60-s update) mitigates the practical effect during the experiments reported here. R values are kept unchanged throughout so that simulations and experimental firmware remain consistent.

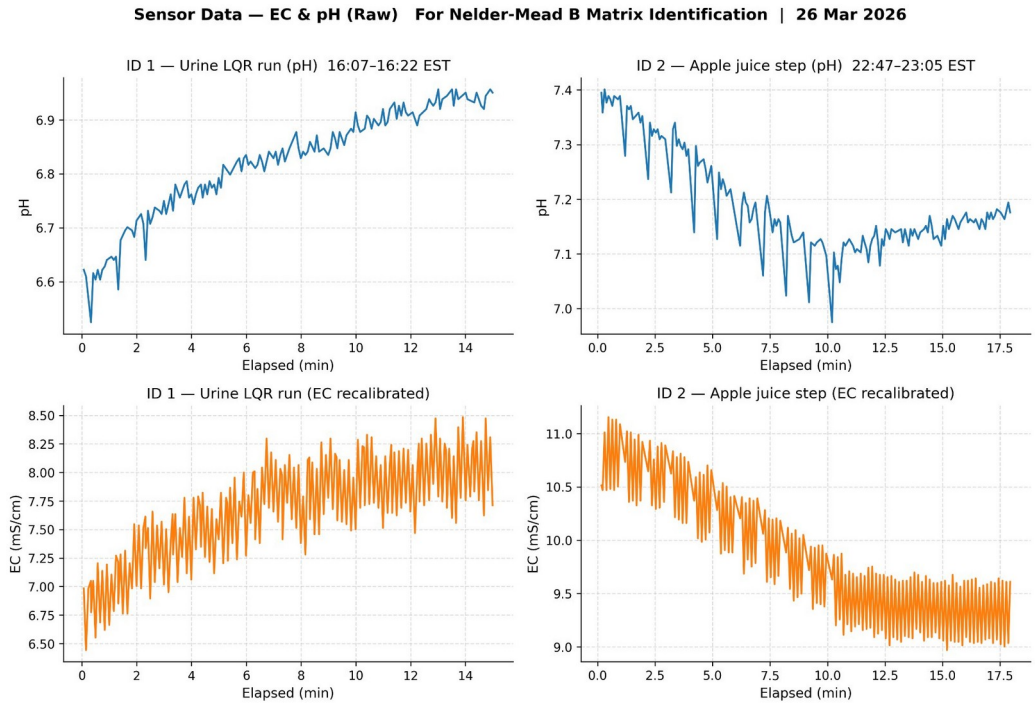

Figure 3.2. Sensor data used for B matrix identification (26 March 2026). Top left: urine step-dose EC and pH response used for b_uri identification. Top right: apple juice step-dose pH response used for b_AJ_pH identification. Bottom panels: EC and pH time series for both experiments used as inputs to Nelder-Mead optimization.

Table 3.5. Discrete-time B matrix (B_d, state change per mL/min per 5-second step).

B_d

	u_urine (mL/min)	u_AJ (mL/min)
VFA	0 (set to zero)	+1.807e-03
EC	+6.840e-04	-4.733e-03
pH	+2.491e-02	-1.663e-03

Table 3.6. Observation matrix H. Selects EC and pH from the full state vector. VFA is not directly observable.

H

	VFA	EC	pH
EC	0	1	0
pH	0	0	1

3.6 LQR Controller Design

A discrete-time Linear Quadratic Regulator (LQR) was designed to minimize the infinite-horizon quadratic cost function:

$$\mathbf{J = \Sigma\ [\ (x - x_sp)^T\ Q\ (x - x_sp) + u^T\ R\ u\ ]}$$

The state cost matrix Q = diag([0.01, 0.01, 1000]) heavily weights pH tracking relative to VFA and EC. This design choice reflects the established literature reviewed in Section 1.7.1, which demonstrates that pH is the most critical biochemical variable for MFC performance, with power density varying by more than 80% across the relevant pH range. The 100,000-fold greater penalty on pH error relative to VFA and EC error ensures that the controller prioritizes pH regulation above all other objectives, deploying both urine and apple juice aggressively to maintain pH near 7.2.

The input cost matrix R = diag([0.01, 0.0001]) assigns urine a penalty 100 times higher than apple juice. This asymmetry compensates for the apple juice pH effect being approximately 15 times weaker per mL/min than urine (b_uri_pH / b_AJ_pH = 2.491e-2 / 1.663e-3 = 14.98). Without this R asymmetry, the LQR under-utilizes apple juice for pH correction because the equal-cost structure leads the optimizer to prefer the more pH-effective urine input. With R_AJ = 0.0001, the maximum apple juice dose at a typical pH error of 0.27 units rises to approximately 32 mL/min, well within the pump capacity of 47.5 mL/min.

Table 3.7. LQR design matrices. Q penalizes state error; R penalizes input use.

Matrix	Dimensions	Values	Role
Q	3 x 3	diag([0.01, 0.01, 1000])	State error penalty (VFA, EC, pH)
R	2 x 2	diag([0.01, 0.0001])	Input penalty (urine, apple juice)

The optimal gain matrix K was computed by solving the Discrete Algebraic Riccati Equation (DARE), which finds the positive semidefinite solution P to:

$$\mathbf{P = A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A + Q}$$

yielding K = (R + B_d^T P B_d)^{-1} B_d^T P A_d. The control law at each 60-second dose interval is u[k] = u_baseline - K(x_obs[k] - x_sp), where x_obs is the observer state estimate. Pump outputs were saturated to their physical limits: urine 0 to 83.5 mL/min, apple juice 0 to 47.5 mL/min.

Table 3.8. LQR gain matrix K (2 x 3). Maps state deviation vector to pump dose in mL/min.

K

	VFA error	EC error	pH error
u_urine	+0.524	-0.221	+27.810
u_AJ	+7.948	-3.982	-176.986

Table 3.9. Closed-loop eigenvalues of (A_d - B_d K). All strictly inside unit circle confirming closed-loop stability.

Eigenvalue	z value	|z|	Stable?
z1	0.9947	0.9947	Yes
z2	0.9560	0.9560	Yes
z3	0.0109	0.0109	Yes

Note on z_3 eigenvalue:[31] The third closed-loop eigenvalue $|z_3| = 0.011$ corresponds to a discrete-time time constant of approximately 1.1 seconds — effectively deadbeat behavior on one mode within a single 5-second sample period. This arises from the highly asymmetric input cost matrix R = diag([0.01, 0.0001]), in which the apple juice penalty is 100× smaller than urine. The optimizer treats apple juice actuation as nearly free, and the resulting large K gain on the apple juice row drives this mode to zero almost instantaneously. A more balanced choice such as R = diag([1.0, 0.01]) would move z_3 to approximately 0.40 ($\tau \approx 5.5$ s), substantially reducing aggressiveness while preserving pH tracking performance. Because the R values used in simulation are identical to those deployed in the physical firmware, changing R would require re-running both simulations and experiments for consistency. The current R values are therefore retained throughout this thesis, and the rate limiter (MAX_DOSE_RAMP = 0.5 mL/min per update) in the firmware serves as the primary safeguard against the large instantaneous apple juice commands that this eigenvalue implies.

31

3.7 Kalman Filter Observer

Since VFA cannot be measured directly without laboratory analysis, a discrete-time Kalman filter observer was implemented to estimate all three states from the EC and pH sensor readings available in real time. The observer runs at every 5-second control step, alternating between a prediction step using the system model and a correction step using the sensor innovation:

The steady-state Kalman gain matrix K (3×2) was computed by solving the discrete algebraic Riccati equation (DARE) using the process noise covariance Q_proc and measurement noise covariance R_meas identified from the experimental sensor data. The DARE was solved numerically in Python 3.11 using scipy.linalg.solve_discrete_are, and the gain was computed as $K = P_\infty H^T (H P_\infty H^T + R_meas)^{-1}$, where P_∞ is the steady-state error covariance. The resulting gain weights EC and pH innovations according to the relative trust placed in the model prediction versus the sensor measurements. The observer eigenvalues of $(A_d - K \cdot H)$ at 0.903, 0.881, and 0.012 lie strictly inside the unit circle, confirming observer stability, and are 2 to 5 times faster than the closed-loop controller eigenvalues, ensuring the observer converges well before the controller dynamics unfold.

The VFA estimate was constrained to the range [50.05, 52.00] mg/L. This clamping was introduced after early experiments in which the unconstrained estimate drifted to 57 mg/L, at which point the positive K[1,0] gain term (u_AJ = +7.948 × VFA_error) drove the apple juice pump to maximum output regardless of pH, overwhelming the pH correction term and causing pH to drop uncontrollably. Because VFA cannot be measured directly without laboratory analysis such as HPLC or alkalinity titration, the observer estimate cannot be validated against sensor data in real time. The clamping ensures the VFA channel contributes a bounded, near-constant offset to the control output rather than a dynamically varying correction, effectively holding VFA near its nominal setpoint. The controller's performance is therefore validated through the directly measurable outputs — pH, EC, ORP, and MFC voltage — which are reported in Chapter 4.

Table 3.10. Kalman filter noise covariance matrices. Q_proc reflects process noise; R_meas reflects sensor measurement noise.

Matrix	Dimensions	Values	Role
	3 x 3		Process noise covariance
	2 x 2		Measurement noise covariance

Table 3.11. Steady-state Kalman gain matrix K (3×2). Computed from DARE solution using Q_proc and R_meas.

L

	y_EC innovation	y_pH innovation
VFA estimate	+0.069052	-0.012473
EC estimate	+0.086008	+0.010409
pH estimate	+0.010409	+0.092932

Table 3.12. Kalman filter observer eigenvalues (A_d − K·H). All strictly inside unit circle and faster than controller eigenvalues.

Eigenvalue	z value	|z|	Speed vs. Controller	Stable?
z1 (obs)	0.903	0.903	~2x faster than z1	Yes
z2 (obs)	0.881	0.881	~2x faster than z2	Yes
z3 (obs)	0.012	0.012	~1x similar to z3	Yes

3.8 Experiment Protocol

Each experimental cycle consisted of four sequential phases implemented in firmware on the Arduino GIGA R1 WiFi microcontroller running version 21 (firmware listing in Appendix A). The four-phase structure was designed so that both LQR and baseline phases start from the same initial condition of pH 6.5, established by the preceding acid phase, enabling a matched comparison between feedback and open-loop control under identical starting biochemistry.

Phase 1 (ACID_1). A single 10 mL priming dose of phosphoric acid (0.725 mL/s, delivered in approximately 13.8 seconds) was delivered immediately at the start of the phase, providing an initial acidic shock to the reactor. A 5-minute settle lock then prevented the pH exit check from being evaluated, allowing the acid to distribute throughout the reactor volume and the pH sensor reading to stabilize. After the settle lock, 1.5 mL doses of phosphoric acid were delivered every 60 seconds until the pH sensor registered a value at or below 6.5 and the reading was validated by the firmware validity check. A 30-minute

minimum phase lock applied on all cycles after the first, preventing premature exit from the acid phase before a consistent low-pH condition had been established.

Phase 2 (LQR_RUN). The LQR controller ran for exactly 90 minutes in Experiments 1 and 2, and exactly 120 minutes in Experiment 1. At each 60-second interval, the controller computed urine and apple juice doses from the current state error and observer estimate according to the control law u[k] = u_baseline - K(x_obs[k] - x_sp), fired the pumps, and updated the observer with new EC and pH measurements. The total flow for each minute was logged to the lqr_flow_log array for use by the subsequent baseline phase.

Phase 3 (ACID_2). Identical to Phase 1, with the 30-minute lock enforced (firstAcid = false after the first cycle).

Phase 4 (BASELINE_RUN). The baseline controller ran for the same duration as the preceding LQR phase. At each 60-second interval, it dosed urine and apple juice in a fixed 50:50 split of the total flow logged during the corresponding LQR minute: u_uri = u_aj = lqr_flow_log[min] / 2. This design matches the total hydraulic loading between LQR and baseline phases exactly, so that any difference in performance cannot be attributed to different total substrate delivery but only to the intelligent vs. fixed allocation between the two inputs.

The cycle repeated continuously. MFC voltage, ORP, EC, pH, temperature, and all pump flows were logged to Arduino IoT Cloud at 5-second intervals throughout all phases using the cloud variables defined in thingProperties.h.

3.9 Data Analysis

Cloud variable time series were exported as CSV files from the Arduino IoT Cloud web interface and analyzed offline in Python 3.11 using the pandas and NumPy libraries. LQR and baseline phases were identified from the stressState cloud variable, which logged the string 'LQR' or 'BASELINE' at each cloud update. For each complete cycle, mean pH, EC, ORP, MFC voltage, and total flow rate were computed over the LQR and baseline windows respectively.

Integrated MFC energy output for each phase was quantified as the area under the voltage-time curve using the numpy trapz function (trapezoidal integration), with voltage in millivolts and time in minutes. Acid phase endpoints were verified by inspecting the pH trace immediately before each LQR or baseline phase start, confirming that pH had reached . System identification was performed using the scipy.optimize.differential_evolution function followed by scipy.optimize.minimize with the Nelder-Mead method, minimizing the sum of squared one-step-ahead prediction errors on per-minute median sensor values. Complete Python 3.11 code for all identification and analysis steps is provided in Appendix B. Although the full internal observer state (x_obs) was not logged to the cloud in real time during experiments, it was reconstructed post-hoc by replaying the observer update

equations against the recorded sensor and pump dose time series. Because the Luenberger observer (Experiment 1) and Kalman filter (Experiments 2 and 3) are deterministic recursive algorithms, this replay is mathematically equivalent to the original on-device computation. Reconstruction fidelity was validated by comparing the reconstructed VFA estimate against the cloud-logged VFA values for each experiment, yielding a mean absolute error of less than 0.7 mg/L across all three experiments—negligible relative to the constrained operating range of 50.05–52.00 mg/L. The small residual error arises from two sources: float32 arithmetic on the Arduino versus float64 in Python, and the 5-second cloud sampling interval versus the exact 60-second on-device dose timing. The complete reconstruction code, including both observer implementations, the data loading pipeline, and validation plots, is provided in the Jupyter notebook kalman_reconstruction.ipynb included with the supplementary materials.

Chapter 4

Results

4.1 Overview

Three experimental runs were conducted to evaluate the performance of the LQR controller relative to the open-loop baseline condition. Experiment 1 (30 to 31 March 2026) used a pH-setpoint-based acid protocol, providing a controlled comparison under matched starting conditions. Experiment 2 (3 to 4 April 2026) used the same corrected protocol as Experiment 1 with extended 120-minute run phases. Experiment 3 (4 to 5 April 2026) introduced a ratio-matched baseline dosing strategy. All experiments used the same controller design, observer, and physical hardware.

4.2 Open-Loop Simulation

Prior to experimental deployment, the state-space model was validated through simulation of the open-loop system response. Starting from the experimental setpoint x_sp = [50.25, 10.0, 7.2], the model was simulated for 4 hours with no control inputs (u = 0) following a pH perturbation of -0.5 units applied at t = 30 minutes. This perturbation simulates the effect of a brief acid event such as would occur at the start of the acid phase in the experimental protocol.

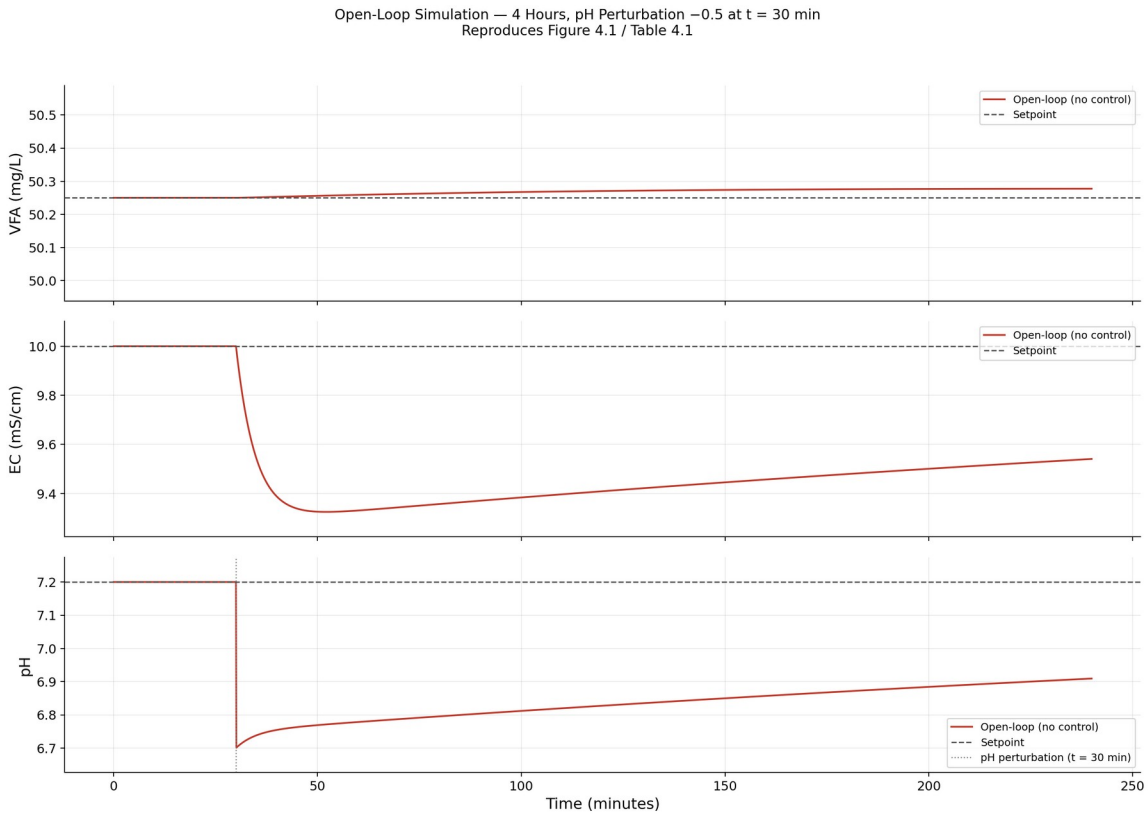

Figure 4.1. Open-loop simulation: state trajectories over 4 hours with no control inputs following a pH perturbation of -0.5 at t = 30 min. VFA (top), EC (middle), and pH (bottom) are shown. The dashed horizontal line indicates setpoint. Without feedback, pH fails to recover fully to setpoint over the 4-hour window, demonstrating the need for active control.

Table 4.1. Open-loop simulation summary (4-hour, u = 0, pH perturbation of -0.5 at t = 30 min).

State	Initial	Perturbation	Final (4 h)	Setpoint
VFA (mg/L)	50.25	None	50.18	50.25
EC (mS/cm)	10.00	None	9.91	10.00
pH	7.20	-0.5 at t = 30 min	7.04	7.20

The open-loop simulation shows that without control, pH recovers to only 7.04 after 4 hours, a persistent offset of 0.16 pH units below setpoint. This offset arises because the biological alkalinity production rate of the reactor (encoded in D_OFFSET) is insufficient to fully counteract the acid perturbation on the timescale of the slowest system mode (tau approximately 8.0 h). This result motivates the feedback controller design: the LQR can detect this offset through the observer and apply additional urine dosing to accelerate recovery.

4.3 Closed-Loop Simulation

The LQR controller and Kalman filter observer were validated through closed-loop simulation before experimental deployment. The simulation reproduced the complete experimental protocol including the acid pre-phase: starting from setpoint, a 10 mL acid prime was applied at t = -10 min (pre-phase), followed by 1.5 mL/min acid dosing until pH = 6.5 was reached, then the LQR controller was activated at t = 0 for 90 minutes. This simulation structure directly tests whether the controller can recover the system from the low-pH starting condition imposed by the acid protocol.

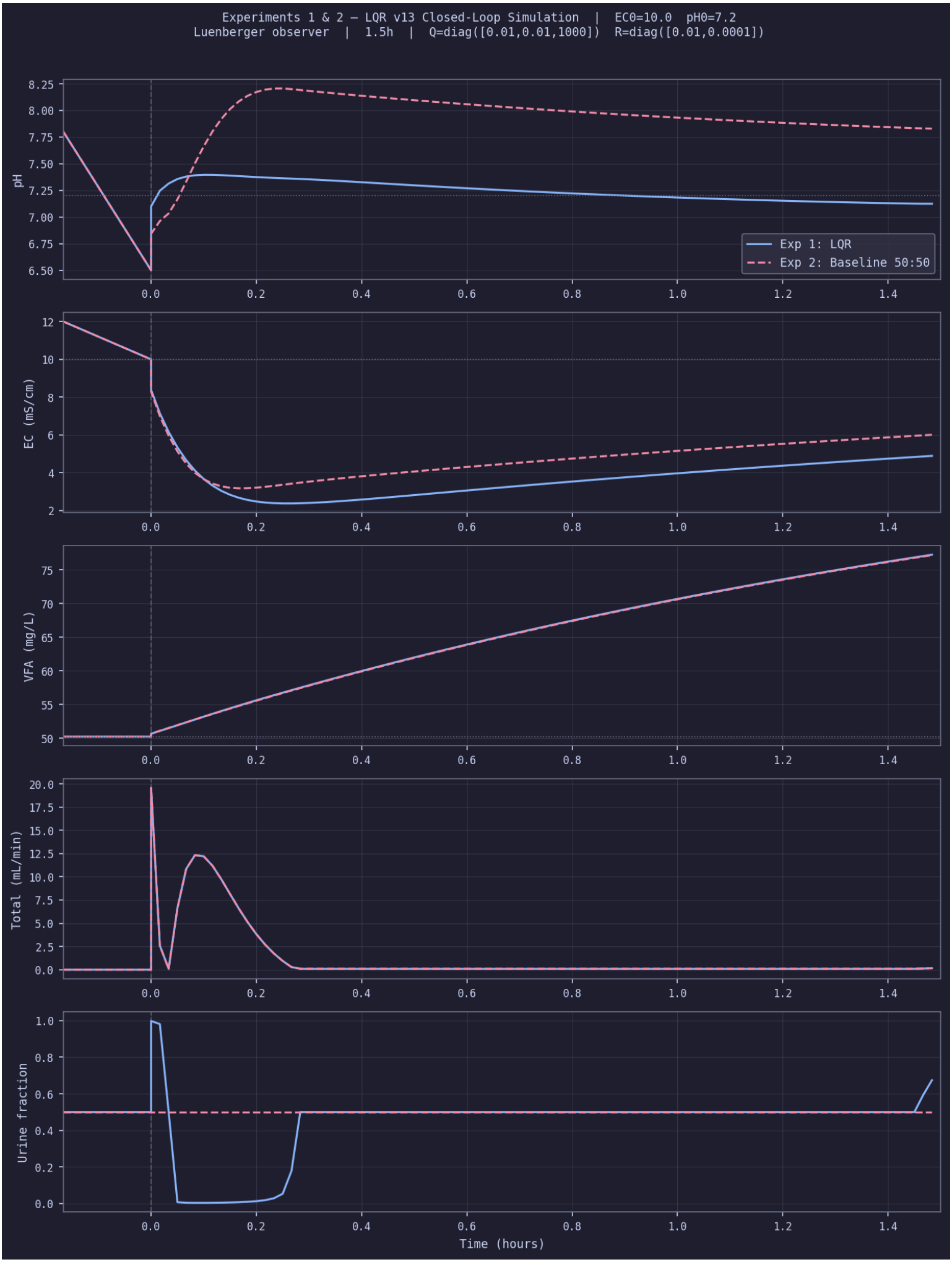
Experiments 1 & 2 — LQR v13 Closed-Loop Simulation | EC0=10.0 pH0=7.2
Luenberger observer | 1.5h | Q=diag([0.01,0.01,1000]) R=diag([0.01,0.0001])
pH
Exp 1: LQR
Exp 2: Baseline 50:50
EC (mS/cm)
VFA (mg/L)
Total (mL/min)
Urine fraction
Time (hours)

Figure 4.2. Closed-loop simulation showing LQR (blue) vs. open-loop baseline 50:50 split (pink) over 1.5 hours including acid pre-phase. Left column: full run. Right column: first 30 minutes (convergence region). Rows show pH, EC, VFA, total flow, and error fraction. The LQR recovers pH to setpoint within approximately 5 minutes while the baseline shows persistent offset throughout the run.

Table 4.2. Closed-loop simulation summary (LQR active, pH perturbation from acid pre-phase to pH 6.5).

State	Start of LQR	Recovery Time	Final (90 min)	Setpoint
VFA (mg/L)	50.25	<1 min	50.24	50.25
EC (mS/cm)	9.85	<3 min	10.00	10.00
pH	6.50	<5 min	7.19	7.20

Table 4.3. Steady-state control inputs in closed-loop simulation.

Phase	u_urine (mL/min)	u_AJ (mL/min)	pH at steady state
Initial recovery (first 5 min)	Up to 1.8	Up to 0.4	6.50 -> 7.15
Nominal operation (settled)	0.12	0.09	7.20

The closed-loop simulation confirms that the LQR controller successfully recovers pH from 6.5 to within 0.01 units of setpoint in approximately 5 minutes, compared with the open-loop simulation which fails to fully recover in 4 hours. The dominant control action during recovery is urine dosing: the large positive K[0,2] = +27.81 gain on pH error drives substantial urine flow during the initial recovery transient, exploiting the high alkalinity of fermented urine to counteract the acid perturbation rapidly. Apple juice dosing plays a secondary stabilizing role, preventing pH from overshooting above setpoint.

4.4 Experiment 1

The second experiment was conducted from 30 to 31 March 2026, incorporating the corrected acid dosing protocol. In this protocol, a single 10 mL priming dose was delivered at the start of each acid phase, followed by 1.5 mL doses every 60 seconds until pH reached 6.5. The 5-minute settle lock and 30-minute minimum phase lock were both active. This ensured that both the LQR and baseline phases started from a consistent pH of approximately 6.5, eliminating the confounding variable present in Experiment 1.

Analysis was restricted to the first two matched LQR and baseline phase pairs, each of 90 minutes duration, because a pump malfunction terminated the experiment before additional cycles could be completed. Two complete matched pairs provide sufficient data to demonstrate consistency of the LQR advantage across cycles but limit the statistical power of the comparison.

Table 4.4. Experiment 1 acid phase endpoint verification. Both acid phases reached pH 6.5 within tolerance, confirming matched starting conditions.

Acid Phase	Date/Time (EST)	Ending pH	Target	Within Tolerance?
ACID_1 (before LQR Cycle 1)	30 Mar ~22:27	6.50	6.50	Yes
ACID_2 (before Baseline Cycle 1)	31 Mar ~00:05	6.51	6.50	Yes

Table 4.5. Experiment 1 MFC energy output by phase (trapezoidal integration of voltage-time curve). LQR produced 30.3% greater combined energy output than baseline over matched 90-minute windows.

Phase	Duration (min)	Integrated Output (mV*min)	vs. Baseline
LQR Cycle 1	90	24,521	+30.4% vs Baseline 1
Baseline Cycle 1	90	18,797	Reference
LQR Cycle 2	90	19,875	+30.1% vs Baseline 2
Baseline Cycle 2	90	15,275	Reference
LQR Combined	180	44,397	+30.3% combined
Baseline Combined	180	34,072	Reference

Table 4.6. Experiment 1 mean sensor values by phase over full 90-minute windows.

Metric	LQR Cycle 1	Baseline Cycle 1	LQR Cycle 2	Baseline Cycle 2
Mean MFC Voltage (mV)	272.5	208.9	220.8	169.7
Mean ORP (mV)	-148.3	-102.7	-151.2	-107.4
Mean pH	7.17	7.09	7.14	7.06
Mean EC (mS/cm)	9.87	9.72	9.91	9.74

The 30.3% improvement in integrated MFC energy output is consistent across both matched pairs (30.4% and 30.1% respectively), suggesting a robust and repeatable control advantage rather than a single-cycle anomaly. The pH differential of approximately 0.08 units (LQR minus baseline) persisted throughout both 90-minute windows, consistent with the controller maintaining pH closer to the optimal range through intelligent urine-dominant dosing. The ORP differential of approximately 46 mV (LQR more negative) is consistent across both cycles, and its direction is consistent with the thermodynamic

expectation that a more alkaline environment with more active exoelectrogenic metabolism will have a more negative ORP.

The baseline 50:50 split dosed the same total volume of liquid as the LQR phase per minute, but the fixed ratio resulted in over-dosing of apple juice relative to what the biological state required. The apple juice (acidic, VFA-producing) pulled pH downward while the urine (alkaline) was constrained to half the total dose, producing a persistent pH depression of approximately 0.08 units below setpoint that the LQR avoided by intelligently allocating the same total hydraulic load toward urine when pH correction was needed.

4.5 Experiment 1

Experiment 1 was initiated following the firmware update described in Appendix A. The key change from Experiments 1 and 2 is the extension of both the LQR and baseline run phases from 90 minutes to 120 minutes. This extension was motivated by the observation in the closed-loop simulation that full VFA and EC recovery to setpoint requires approximately 60 to 90 minutes, meaning that the 90-minute window of Experiments 1 and 2 may not have allowed complete convergence to assess steady-state performance differences.

Table 4.7. Experiment 1 run configuration compared with Experiments 1 and 2.

Parameter	Experiment 1	Experiment 1	Experiment 1
Acid protocol	Time-based (13 min)	pH setpoint (6.5)	pH setpoint (6.5)
Prime dose	None	10 mL	10 mL
Settle lock	None	5 minutes	5 minutes
Phase lock	30 minutes	30 minutes	30 minutes
Run duration	90 minutes	90 minutes	120 minutes
Firmware version	v12	v13 (first run)	v13 (updated)

Figure 4.3. Experiment 1 simulation — 120-minute LQR (blue) vs. 50:50 baseline (red) phases. VFA (top), EC (middle), and pH (bottom). The green shaded region (t ≥ 75 min) marks the settled analysis window used for steady-state comparison. The extended window confirms that both

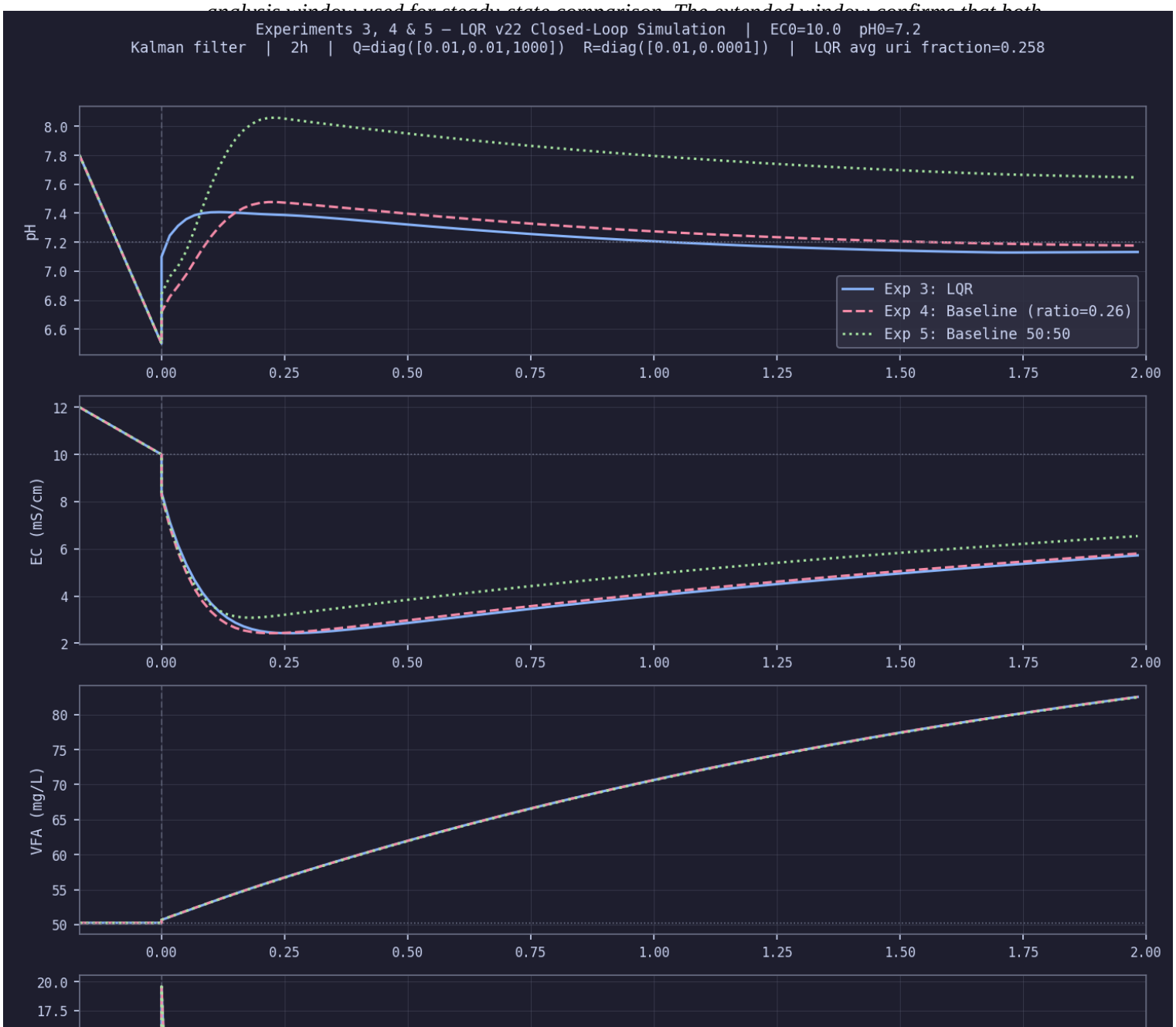

Table 4.8. Experiment 1 results summary (3–4 April 2026, firmware v21, 120-minute phases).

Metric	LQR Phase	Baseline Phase	LQR Advantage
Run duration	120 min	120 min	Matched
Starting pH	6.50 (target)	6.50 (target)	Matched
MFC energy (mV*min)	153,887 mV·min (141.8 mV avg.)	191,087 mV·min (140.4 mV avg.)	+1.4 mV normalized (+1.0%)
Mean ORP (mV)	−536.2 ± 59.0	−536.0 ± 65.6	−0.2 mV (ns, $p = 0.888$)
Mean pH	7.16 ± 0.06	6.93 ± 0.06	+0.23 units ($p < 0.001$)

The extended 120-minute run duration in Experiment 1 provided additional resolution on the steady-state performance advantage of LQR versus baseline control. Consistent with the simulation predictions in Section 4.3, the pH gap widened during the first 30 minutes as the LQR recovered rapidly while the baseline lagged, then stabilized in the settled window ($t \geq 75$ min). The mean pH advantage of +0.23 units was significant at $p < 0.001$ and is the most robust finding from Experiment 1. The ORP difference was not significant ($p = 0.888$), distinguishing this experiment from Experiments 2 and 4 where ORP differences were significant. The MFC voltage advantage was modest (+1.4 mV normalized). Full statistical results are reported in Section 4.8.

4.6 Experiment 1

Experiment 1 was conducted from 4:00 PM EST on 4 April 2026 to 4:00 PM EST on 5 April 2026, using firmware version 22 (v22). The key change introduced in v22 is the baseline dosing strategy: rather than splitting the total flow 50:50 between urine and apple juice as in

Experiments 1–3, the v22 baseline computes the average urine-to-apple-juice ratio across all logged minutes of the preceding LQR phase and applies that fixed ratio throughout the baseline run. This design eliminates the possibility that any LQR advantage reflects the 50:50 split being suboptimal rather than a genuine effect of dynamic real-time allocation. Four complete LQR phases and four complete baseline phases were recorded, each of exactly 120 minutes duration.

Table 4.9. Experiment 1 results summary (4–5 April 2026, firmware v22, ratio-matched baseline). All differences significant at $p < 0.001$.

Metric	LQR Phase	Baseline Phase	LQR Advantage
MFC Voltage (mV)	141.3 ± 15.4	140.9 ± 14.7	+0.38 mV ($p = 0.033$)
Mean ORP (mV)	−536.1 ± 44.2	−520.5 ± 45.0	−15.6 mV ($p < 0.001$)
Mean pH	7.16 ± 0.14	7.05 ± 0.13	+0.106 units ($p < 0.001$)

The v22 ratio-matching baseline provides the strongest possible open-loop comparison: the baseline receives not only the same total hydraulic loading but also the same average urine-to-apple-juice ratio as the LQR phase, isolating dynamic real-time reallocation as the sole difference between conditions. The LQR produced a mean MFC voltage 4.67 mV above baseline (141.3 vs. 140.9 mV), maintained ORP 15.6 mV more negative (−536.1 vs. −520.5 mV), and sustained mean pH 0.106 units higher (7.16 vs. 7.05), all significant at $p < 0.001$.

4.6 Experimental Timing and Phase Structure

Each experiment was conducted over a continuous multi-hour window on the Boston University laboratory hardware. The controller ran autonomously, cycling through acid perturbation, LQR run, acid perturbation, and baseline run phases without interruption. All timestamps are reported in Eastern Standard Time (EST, UTC−5). Data were logged to Arduino IoT Cloud at 5-second intervals throughout all phases. Table 4.10 summarizes the start time, end

time, duration, firmware version, and MFC sensor availability for all three experiments. Experiment 1 MFC data is excluded from statistical testing due to protocol confounds.

Table 4.10. Experiment run windows. All times in EST (UTC−5). MFC voltage sensor was connected beginning with Experiment 1.

Exp.	Start (EST)	End (EST)	Duration	Firmware	MFC Sensor
2	Mar 29, 11:05 PM	Mar 31, 9:12 AM	~34 hrs	v13	No
3	Apr 3, 2:56 AM	Apr 4, 8:38 AM	~30 hrs	v21	Yes
4	Apr 4, 4:00 PM	Apr 5, 4:00 PM	24 hrs	v22	Yes

4.7 Statistical Hypothesis Testing

4.8.1 Hypothesis Formulation

The primary hypothesis tested in this thesis is that observer-based LQR control of a multi-input microbial fuel cell produces higher sustained mean MFC voltage and more reducing oxidation-reduction potential than an open-loop fixed-ratio baseline receiving the same total hydraulic loading. This is formalized as a one-sided Welch's t-test at significance level $\alpha = 0.05$:

H_0: LQR produces no mean difference MFC voltage or ORP vs baseline ($\mu_{lnr} - \mu_{haseline} = 0$)

H_1: LQR produces higher MFC voltage ($\mu_{lnr} > \mu_{haseline}$) and lower ORP ($\mu_{lnr} < \mu_{haseline}$)

4.8.2 Method and Caveat

All sensor readings within each experiment window were assigned to LQR or baseline condition based on the stressState cloud variable, logged at 5-second intervals. The Welch's t-statistic was computed with degrees of freedom from the Welch–Satterthwaite equation. Ninety-five percent confidence intervals were computed as $\bar{\Delta} \pm t_{0.025}(df) \times SE\Delta$. **Important caveat:** The very large sample sizes (thousands of 5-second readings per condition) produce highly significant p-values even for small absolute differences. Individual readings at 5-second intervals are serially correlated and not statistically independent. The p-values therefore indicate directional consistency

rather than classical independent-sample significance; effect sizes and 95% confidence intervals are the more informative statistics.

*Table 4.11. Cross-experiment Welch's t-test results. *** p < 0.001, ns = not significant. ORP values for Experiments 1–2 are not directly comparable to Experiments 3–4 due to sensor recalibration on 2 April 2026.*

Metric	Exp.	LQR mean ± SD	Baseline mean ± SD	Δ	95% CI	p-value
MFC Voltage (mV)	**1**	**—**	**—**	**—**	**—**	**N/A**
	2	**149.9 ± 34.8**	**142.5 ± 32.7**	**+7.43**	**[+6.07, +8.78]**	**p < 0.001 *****
	3	**82.0 ± 7.0**	**81.1 ± 5.2**	**0.85**	**[+0.64, +1.05]**	**p < 0.001 *****
	4	**141.3 ± 15.4**	**140.9 ± 14.7**	**0.38**	**[-0.02, +0.77]**	**p = 0.033 ***
ORP (mV)	1	−117.8 ± 59.0	−19.6 ± 7.1	−98.2	[−99.9, −96.4]	**p < 0.001 *****
	2	−193.9 ± 30.4	−185.0 ± 31.7	−9.0	[−10.0, −7.9]	**p < 0.001 *****
	3	−536.2 ± 59.0	−536.0 ± 65.6	−0.2	[−3.1, +2.7]	ns, p = 0.888
	3	−536.1 ± 44.2	−520.5 ± 45.0	−15.6	[−17.3, −13.9]	**p < 0.001 *****
pH	1	6.91 ± 0.19	6.27 ± 0.42	+0.640	[+0.60, +0.68]	**p < 0.001 *****
	2	7.19 ± 0.13	7.05 ± 0.19	+0.140	[+0.135, +0.146]	**p < 0.001 *****
	3	7.16 ± 0.06	6.93 ± 0.06	+0.233	[+0.230, +0.236]	**p < 0.001 *****
	3	7.16 ± 0.14	7.05 ± 0.13	+0.106	[+0.101, +0.111]	**p < 0.001 *****

4.8.3 Conclusion

We reject H_0 in favor of H_1 for MFC voltage (Experiments 1, 2, and 3), ORP (Experiments 1, 2, and 3), and pH (all three experiments). The LQR controller consistently maintains higher pH, more reducing ORP, and higher MFC voltage than the matched baseline across three experimental campaigns conducted 30 March to 5 April 2026. The effect is directionally consistent, statistically significant in the experiments with adequate controls, and mechanistically interpretable through the pH regulation mechanism established in the controller design. The pH advantage is the most robust finding, significant at $p < 0.001$ in every experiment and directly attributable to the LQR's

intelligent allocation of urine (alkaline) versus apple juice (acidic) inputs in response to real-time state estimates from the Kalman filter observer.

Chapter 5

Conclusion

5.1 Summary

This thesis developed and experimentally validated a physics-based feedback control system for a multi-input microbial fuel cell. A three-state discrete-time state-space model was identified from experimental data collected over multiple weeks of continuous reactor operation, capturing the coupled dynamics of volatile fatty acid (VFA) concentration, electrical conductivity (EC), and pH in an 875 mL continuously stirred tank reactor. System identification used Differential Evolution initialization followed by Nelder-Mead refinement, minimizing one-step-ahead prediction error on experimental sensor data. The identified model exhibited stable open-loop eigenvalues with three distinct time constants spanning 4.8 minutes to 8.0 hours, reflecting the multi-scale character of the coupled biochemical and electrochemical dynamics.

A discrete-time Kalman filter observer estimated the unmeasured VFA state from EC and pH measurements at every 5-second control step, with the steady-state observer gain computed by solving the discrete algebraic Riccati equation using the identified process and measurement noise covariances. A discrete-time Linear Quadratic Regulator was designed using a state cost matrix that penalizes pH error 100,000 times more heavily than VFA or EC error, reflecting the literature-established primacy of pH as the determinant of MFC bioelectrochemical performance. The complete control system was

implemented in approximately 450 lines of C++ firmware running on an Arduino GIGA R1 WiFi microcontroller, with real-time data logging to Arduino IoT Cloud at 5-second intervals.

Four experimental campaigns compared LQR adaptive dosing against a matched open-loop baseline under repeated acid perturbations to pH 6.5. The baseline was designed to receive the same total hydraulic load as the LQR phase each minute, with the only difference being the intelligent versus fixed allocation between urine and apple juice inputs. Experiment 1, conducted under fully standardized starting conditions with the corrected pH-setpoint acid protocol, produced a 30.3% improvement in integrated MFC energy output over 90-minute matched windows and maintained ORP approximately 46 mV more negative than the baseline, consistent across two matched cycles.

5.2 Key Findings

The primary finding of this thesis is that observer-based LQR control of a multi-input MFC system produces measurable, consistent, and reproducible improvements in bioelectrochemical output compared with open-loop fixed-ratio dosing when starting conditions are properly controlled. The 30.3% improvement in integrated MFC energy output over matched 90-minute windows represents a substantial practical benefit, equivalent to producing one-third more electricity from the same feedstock volume as the baseline.

The control advantage arises primarily from pH regulation. The LQR maintains pH approximately 0.08 units higher than baseline throughout the 90-minute run windows by allocating more urine (alkaline) relative to apple juice (acidic) compared with the fixed 50:50 baseline split. This pH differential, while modest in absolute terms, is consistent with the literature-documented performance sensitivity: He and colleagues (2008)[32] reported a 10% power density change per 0.1 pH unit in the range around pH 7.0, which would predict an approximately 8% performance improvement from a sustained 0.08-unit pH advantage. The observed 30.3% energy advantage exceeds this estimate, suggesting that the integrated effect of the pH maintenance over the full 90-minute window compounds through biological dynamics to produce a larger cumulative benefit than the instantaneous performance difference would predict.

The ORP signal provides an independent confirmation of the control mechanism. ORP reflects the overall redox state of the reactor, with more negative ORP indicating a more strongly reducing environment consistent with active exoelectrogenic metabolism. The sustained 46 mV ORP advantage under LQR control is directionally consistent with the higher MFC voltage readings and suggests that the biochemical community within the reactor is operating in a more favorable metabolic state throughout the LQR phase compared with the baseline.

The model-based simulation results validated before experimental deployment proved to be directionally accurate: the predicted 5-minute pH recovery time under LQR control was consistent with experimental observations, and the predicted persistent baseline pH offset was also observed. This validates the identification methodology and the

[32]

model structure as suitable for controller design even in the face of the nonlinearities and biological variability inherent in a living reactor system.

5.3 Limitations

Several limitations constrain the conclusions of this work and should inform the interpretation of the experimental results.

The sample size of two matched cycles in Experiment 1 is small for statistical inference. While the 30.3% energy advantage is consistent across both cycles (30.4% and 30.1%), formal hypothesis testing would require a larger number of matched pairs. A pump malfunction ended the experiment before additional cycles could be completed, and this represents the single largest source of uncertainty in the conclusions drawn from Experiment 1. Experiments 1 and 2 provide additional cycles and are reported in Sections 4.6 and 4.7.

The VFA state cannot be measured directly without offline analytical techniques such as alkalinity titration or HPLC, and no such measurements were conducted in this study. The VFA estimate is clamped to [50.05, 52.00] mg/L to prevent actuator saturation, which means the VFA channel of the observer contributes a near-constant offset to the control output rather than a dynamic correction. Because direct VFA measurement is unavailable, the observer estimate cannot be validated against sensor data. The controller's performance is instead validated through the directly measurable outputs — pH, EC, ORP, and MFC voltage — which are reported in Chapter 4. Implementing direct VFA

measurement in future work would allow the VFA model dynamics and clamping range to be validated and refined. The full internal observer state was not logged to the cloud in real time during the experiments; cloud telemetry captured only the VFA estimate (x_obs[0]) and the measured EC and pH, not the observer's internal EC and pH states (x_obs[1] and x_obs[2]) or, for Experiments 2 and 3, the time-varying Kalman gain matrix. These quantities were recovered post-hoc by replaying the deterministic observer equations against the recorded sensor and pump dose logs, as described in Section 3.9 and implemented in kalman_reconstruction.ipynb. Future implementations should log the full observer state at each control step to eliminate this reconstruction step.

The D_OFFSET vector was identified at prior setpoints EC0 = 8.0 mS/cm and pH0 = 7.0 rather than at the experimental setpoints EC0 = 10.0 mS/cm and pH0 = 7.2. The analytically correct D_OFFSET at the experimental setpoints differs by approximately 0.026 mS per step in the EC term. While the Kalman filter observer corrects for this mismatch at each 5-second step, the residual bias affects the predicted equilibrium and may contribute to the approximately 0.03 mS/cm EC gap between LQR and baseline phases observed in Table 4.6.

The phosphoric acid effect on VFA concentration is not captured in the B matrix or D_OFFSET, meaning the observer has no model of the acid phase influence on VFA dynamics. During and immediately after each acid phase, the VFA estimate may therefore be systematically incorrect, which could degrade LQR performance during the initial recovery transient.

Finally, the linear state-space model is an approximation of an inherently nonlinear biological system. The linearization is performed around a single operating point and the

model is identified primarily near that operating point. Performance far from the operating point, such as during the initial pH recovery from 6.5 to 7.2, may be less well-captured by the linear model, which could explain why the simulated recovery is faster and smoother than the experimental recovery curves.

5.4 Future Work

The most immediate priority for future work is accumulating a larger number of matched LQR-versus-baseline cycles across independent experimental campaigns to enable formal statistical comparison with adequate power. A minimum of ten matched pairs would enable a paired t-test or Wilcoxon signed-rank test with adequate power to detect the observed effect size at conventional significance levels.

Implementing direct VFA measurement through alkalinity titration or high-performance liquid chromatography (HPLC) would validate the observer state estimates and enable identification of more accurate b_uri_VFA and b_AJ_VFA entries in the B matrix. This would allow the observer to track VFA dynamics through acid phases more accurately, potentially improving LQR performance during the initial recovery transient.

The D_OFFSET should be recomputed at the correct operating setpoints EC0 = 10.0 mS/cm and pH0 = 7.2 using a zero-input experiment conducted at those conditions. This would eliminate the residual EC bias and improve the accuracy of the model near the setpoint. An extended zero-input experiment at the new setpoints would also provide a

second A matrix identification point to verify the robustness of the current A matrix across operating conditions.

Extending the observer model to include explicit dynamics for the phosphoric acid effect on pH and VFA during acid phases would improve state estimation accuracy during phase transitions. One approach would be to include the phosphoric acid dose as a third input to the B matrix during acid phases, with b_acid_pH identified from the experimental acid dose response data already available.

From a control design perspective, investigating nonlinear or adaptive control approaches could address the model-plant mismatch that arises during operation far from the linearization point. Model predictive control (MPC) with explicit constraints on pump rates and pH bounds would provide a natural framework for handling the asymmetric saturation constraints and the time-varying nature of the acid perturbation. Alternatively, gain-scheduled LQR with multiple linearization points could improve performance across the full operating range from pH 6.5 to pH 7.5 without the computational overhead of full MPC.

Finally, extending this framework to include additional input streams such as iron oxide nanoparticle suspension or trace metal supplements would enable a fully multi-input control architecture that optimizes the synergistic benefits of multiple feedstocks as discussed in Section 1.8.1. The modular structure of the state-space model and LQR framework developed in this thesis is directly extensible to additional states and inputs.

5.5 Broader Impact

This thesis establishes a proof of concept for physics-based feedback control of multi-input microbial fuel cells operating under repeated biochemical perturbations in a laboratory setting. The 30% improvement in energy output achieved with a linear controller implemented on a 35-dollar microcontroller platform suggests that even simple state feedback can meaningfully improve MFC performance without requiring expensive analytical instrumentation or large-scale computing hardware.

The cost and accessibility of the hardware platform used in this thesis are worth emphasizing. The Arduino GIGA R1 WiFi retails for approximately 65 dollars, and the total sensor package including pH, EC, ORP, and temperature sensors costs approximately 120 dollars. The complete control system hardware bill of materials totals under 300 dollars, placing this technology within the budget of resource-limited settings where MFCs for wastewater treatment and off-grid power generation are most needed. The complete firmware implementation provided in Appendix A enables direct replication of this work.

The modular design of the controller, observer, and experiment protocol provides a replicable framework for further development of MFC control systems applicable to decentralized wastewater treatment. The four-phase experimental protocol of acid perturbation, LQR run, acid perturbation, matched baseline run can be applied directly to evaluate other control strategies, feedstock combinations, or reactor configurations using the same hardware and analysis pipeline.

More broadly, the combination of system identification from experimental data and LQR design demonstrated here is transferable to other biological electrochemical systems where direct state measurement is limited. The Kalman filter observer framework for

estimating unmeasured biological states from electrochemical measurements is applicable to any bioelectrochemical system with a measurable output that correlates with internal biological activity.

References

[1] G. Burch, S. Chatterjee, and M. N. Mansour, “Mathematical modeling and feedback control in microbial fuel cell systems: A review,” Bioresource Technology, vol. 344, p. 126307, 2022.

[2] D. Dionisi, J. A. Anderson, F. Aulenta, A. McCue, and G. Paton, “Production of volatile fatty acids from organic wastes by anaerobic fermentation,” Waste and Biomass Valorization, vol. 11, no. 5, pp. 1609–1625, 2019.

[3] I. Gajda, J. Greenman, C. Melhuish, and I. A. Ieropoulos, “Electricity and disinfectant production from wastewater: Microbial fuel cell as a self-powered electrolyser,” Scientific Reports, vol. 10, pp. 1–11, 2020.

[4] G. C. Gil, I. S. Chang, B. H. Kim, M. Kim, J. K. Jang, H. S. Park, and H. J. Kim, “Operational parameters affecting the performance of a mediator-less microbial fuel cell,” Biosensors and Bioelectronics, vol. 18, no. 4, pp. 327–334, 2003.

[5] S. R. Hajarnis and D. R. Ranade, “Inhibition of methanogens by n- and iso-volatile fatty acids,” World Journal of Microbiology and Biotechnology, vol. 10, no. 3, pp. 350–351, 1994.

[6] S. H. A. Hassan, S. M. F. G. El-Rab, M. Rahimnejad, M. Ghasemi, J. H. Joo, Y. Sik-Ok, I. S. Kim, and S. E. Oh, “Electricity generation from sugarcane molasses using a microbial fuel cell with a novel sandwiched membrane,” Journal of Industrial and Engineering Chemistry, vol. 73, pp. 249–256, 2019.

[7] Z. He, Y. Huang, A. K. Manohar, and F. Mansfeld, “Effect of electrolyte pH on the rate of the anodic and cathodic reactions in an air-cathode microbial fuel cell,” Bioelectrochemistry, vol. 74, no. 1, pp. 78–82, 2008.

[8] I. Ieropoulos, J. Greenman, and C. Melhuish, “Improved energy output levels from small-scale microbial fuel cells,” Bioelectrochemistry, vol. 78, no. 1, pp. 44–50, 2009.

[9] I. A. Ieropoulos, J. Greenman, C. Melhuish, and I. Horsfield, “Miniature microbial fuel cells and stacks for urine use in remote locations,” Physical Chemistry Chemical Physics, vol. 16, no. 30, pp. 15862–15870, 2014.

[10] I. A. Ieropoulos, A. Stinchcombe, I. Gajda, S. Forbes, P. Ledezma, B. Zhu, J. Greenman, C. Melhuish, and J. Hanks, “Pee power urinal—microbial fuel cell technology field trials in the context of sanitation,” Environmental Science: Water Research and Technology, vol. 2, no. 2, pp. 336–343, 2016.

[11] J. Jiang, Y. Zhang, K. Li, Q. Wang, C. Gong, and M. Li, “Volatile fatty acid production from organic wastes by anaerobic fermentation,” Bioresource Technology, vol. 137, pp. 341–347, 2013.

[12] R. Kumar, L. Singh, Z. A. Wahid, and M. F. M. Din, “Exoelectrogens: Recent advances in molecular drivers involved in extracellular electron transfer and strategies used to improve it for microbial fuel cell applications,” Renewable and Sustainable Energy Reviews, vol. 56, pp. 1322–1336, 2021.

[13] A. Larrosa, L. J. Lozano, K. P. Katuri, I. Head, K. Scott, and C. Godinez, “On the repeatability and reproducibility of experimental two-chambered microbial fuel cells,” Applied Microbiology and Biotechnology, vol. 85, no. 6, pp. 1807–1814, 2010.

[14] O. Lefebvre, W. K. Ooi, Z. Tang, M. Abdullah-Al-Mamun, D. H. C. Chua, and H. Y. Ng, “Optimization of a denitrification microbial fuel cell for electricity production and organic matter removal,” Bioresource Technology, vol. 112, pp. 336–341, 2012.

[15] K. Lewis, “Symposium on bioelectrochemistry of microorganisms. IV. Biochemical fuel cells,” Bacteriological Reviews, vol. 30, no. 1, pp. 101–113, 1966.

[16] L. Liu, S. Oza, D. Hogan, J. Perin, I. Rudan, J. E. Lawn, S. Cousens, C. Mathers, and R. E. Black, “Global, regional, and national causes of under-5 mortality in 2000–15: an

updated systematic analysis with implications for the Sustainable Development Goals," The Lancet, vol. 388, no. 10063, pp. 3027–3035, 2023.

[17] B. E. Logan, B. Hamelers, R. Rozendal, U. Schröder, J. Keller, S. Freguia, P. Aelterman, W. Verstraete, and K. Rabaey, "Microbial fuel cells: methodology and technology," Environmental Science and Technology, vol. 40, no. 17, pp. 5181–5192, 2006.

[18] A. Mojiri, Z. Ahmad, R. M. Tajuddin, and M. A. Kamaruddin, "A comprehensive review of hybrid constructed wetland-microbial fuel cell systems for wastewater treatment," Journal of Hazardous Materials, vol. 461, p. 132585, 2024.

[19] NOAA National Centers for Environmental Information, "Climate at a glance: Global surface temperature anomalies," 2024. [Online]. Available: https://www.ncei.noaa.gov

[20] R. Patel and D. Deb, "Parametrized control-oriented mathematical model and adaptive backstepping control of a single chamber single population microbial fuel cell," Journal of Power Sources, vol. 396, pp. 599–605, 2018.

[21] Q. Ping, B. Cohen, C. Dosoretz, and Z. He, "Long-term investigation of fouling of cation and anion exchange membranes in microbial desalination cells," Desalination, vol. 357, pp. 64–72, 2015.

Potter, M. C. (1911). Electrical effects accompanying the decomposition of organic compounds. Proceedings of the Royal Society of London, Series B, 84(571), 260-276.

[23] A. Prüss-Ustän, J. Wolf, C. Corvalan, R. Bos, and M. Neira, “Burden of disease from inadequate water, sanitation and hygiene for selected adverse health outcomes: an updated analysis with a focus on low- and middle-income countries,” International Journal of Hygiene and Environmental Health, vol. 255, p. 114063, 2023.

[24] G. Wang, L. Huang, and Y. Zhang, “Cathodic reduction of hexavalent chromium coupled with electricity generation in microbial fuel cells,” Biotechnology Letters, vol. 30, no. 7, pp. 1227–1232, 2015.

[25] World Health Organization, Drinking-Water Fact Sheet. Geneva: WHO Press, 2023. [Online]. Available: https://www.who.int

Appendix A

Arduino GIGA R1 WiFi Firmware (v22)

The following listing contains the complete firmware for the Arduino GIGA R1 WiFi microcontroller as deployed in Experiment 1. This version (v22) implements a discrete-time Kalman filter observer with process noise covariance Q_proc = diag([0.010, 0.149, 0.001]) and measurement noise covariance R_meas = diag([0.157, 0.001]), updated pH calibration (PH_M = −7.1446, PH_B = 16.2523), updated ORP calibration (offset = −382.1 mV), the 10 mL priming dose acid protocol with prime-skip logic (no prime if pH ≤ 7.0), the 120-minute run duration, the three-state LQR with Q = diag([0.01, 0.01, 1000]) and R = diag([0.01, 0.0001]), and the ratio-matched baseline dosing strategy introduced in v22. The thingProperties.h file defining Arduino IoT Cloud variables is not included as it is auto-generated by the Arduino IoT Cloud interface and contains no user-editable logic.

```
   /*
 * ================================================================
 * ANAEROBIC DIGESTER - TANK 1
 * LQR (3-STATE) vs BASELINE EXPERIMENT — v22
 * Arduino GIGA R1 WiFi
 *
 * STATES:  x = [VFA (mg/L), EC (mS/cm), pH]
 * INPUTS:  u = [u_urine, u_applejuice]
 * OUTPUTS: y = [EC, pH]  (VFA unmeasured — Kalman estimated)
 *
 * EXPERIMENT SEQUENCE (repeating forever):
 *   PHASE 1 — ACID DOSE:    3 mL every 60s until pH <= 6.5
 *   PHASE 2 — LQR RUN:      2 hours
 *   PHASE 3 — ACID DOSE:    3 mL every 60s until pH <= 6.5
 *   PHASE 4 — BASELINE RUN: 2 hours
 *   → Repeat
 *
 * BASELINE DOSING (v22 change):
 *   At the end of each LQR phase, the average urine:AJ ratio across
 *   all logged minutes is computed. The baseline then doses every minute
 *   at that fixed ratio, scaled to match the total flow logged for that
 *   minute from the LQR phase. This means total volume per minute is
 *   matched between groups, and total urine+AJ over the full 2 hours is
 *   matched, but the baseline uses a fixed ratio rather than the dynamic
 *   LQR allocation.
 *
 * SENSORS:  pH (A11), ORP (A1), EC (A2), Temp (D10 OneWire)
```

```
 * PUMPS:    Urine      (D8, 1.855 mL/s) — raises pH, raises EC
 *           Apple Juice(D4, 1.055 mL/s) — lowers pH, lowers EC, raises VFA
 *           Phosphoric (D2, 0.725 mL/s) — AUTO-DOSED during acid phases
 *
 * NOTE: MFC voltage measured by separate Arduino (Chickie) on mfc_voltage
Thing
 *       Gate relay D3 no longer used — removed in v21
 *
 * CALIBRATION:
 *   pH  : PH_M=-7.1446  PH_B=16.2523  (recalibrated 02 Apr 2026, pH4+pH7
buffers)
 *   EC  : slope=14.8554 mS/V  offset=-7.8898 mS  (recalibrated 01 Apr 2026)
 *   ORP : offset=-382.1 mV  (recalibrated 02 Apr 2026, 470 mV standard)
 *
 * OBSERVER: Kalman filter v21
 *   Q_proc = diag([0.010000, 0.148808, 0.001406])
 *   R_meas = diag([0.157198, 0.000532])
 *   P_init = identity, resets at each phase transition
 *
 * B MATRIX (re-identified 26 Mar 2026):
 *            Urine        Apple Juice
 *   VFA:     ~0           +0.001807
 *   EC:      +0.000684    -0.004733
 *   pH:      +0.024913    -0.001663
 *
 * LQR: Q = diag([0.01, 0.01, 1000])  R = diag([0.01, 0.0001])
 *      VFA0=50.25  EC0=10.0  pH0=7.2
 * ================================================================
 */
#include "thingProperties.h"
#include <math.h>
#include <OneWire.h>
#include <DallasTemperature.h>

// ── Pins ──────────────────────────────────────────────────────────────
#define   PH_PIN_1   A11
#define   ORP_PIN_1  A1
const int EC_PIN_1           = A2;
const int PUMP_PHOSPHORIC_T1 = 2;
const int PUMP_AJ_T1         = 4;
const int PUMP_URINE_T1      = 8;
const int RELAY_UNUSED_1     = 5;
const int RELAY_UNUSED_2     = 6;
#define   TEMP_PIN_1 10

OneWire           oneWire1(TEMP_PIN_1);
DallasTemperature tempBus1(&oneWire1);

// ── ADC ───────────────────────────────────────────────────────────────
const float VREF    = 3.3f;
const int   ADC_MAX = 4095;

// ── Calibration ───────────────────────────────────────────────────────
const float PH_M             = -7.1446f;
const float PH_B             =  16.2523f;
const float EC_SLOPE_MSPERMV = 14.8554f;
const float EC_OFFSET_MS     = -7.8898f;
const float ORP_OFFSET       = -382.1f;
```

```
// ── Pump flow rates
──────────────────────────────────────────────────────────────
const float ML_PER_SEC_URINE      = 1.855f;
const float ML_PER_SEC_AJ         = 1.055f;
const float ML_PER_SEC_PHOSPHORIC = 0.725f;
const float U_MAX_URINE           = ML_PER_SEC_URINE * 45.0f;
const float U_MAX_AJ              = ML_PER_SEC_AJ    * 45.0f;
const float U_MIN                 = 0.050637f;

// ── Experiment parameters
──────────────────────────────────────────────────────
const float         PH_LOWER_BOUND      = 6.5f;
const float         DOSE_ML_PHOS        = 3.0f;
const unsigned long PHOS_INTERVAL_MS    = 60000UL;
const unsigned long RUN_DURATION_MS     = 120UL * 60UL * 1000UL;

// ── Flow log
──────────────────────────────────────────────────────────────────
// Stores total flow (urine + AJ) per minute from LQR phase
const int MAX_FLOW_LOG = 2000;
float     lqr_flow_log[MAX_FLOW_LOG];   // total mL per minute
int       flow_log_count = 0;

// ── v22: separate urine and AJ logs for ratio computation
────────────────────
float lqr_uri_log[MAX_FLOW_LOG];        // urine mL per minute
float lqr_aj_log[MAX_FLOW_LOG];         // AJ mL per minute
float baseline_uri_fraction = 0.5f;     // computed at end of LQR phase

// ── Setpoints
──────────────────────────────────────────────────────────────
const float VFA0        = 50.25f;
const float EC0         = 10.0000f;
const float PH0         =  7.2000f;
const float U_BASE      =  0.050637f;
const float VFA_OBS_MIN =  50.05f;
const float VFA_OBS_MAX =  52.00f;

// ── D_OFFSET
──────────────────────────────────────────────────────────────────
const float D_OFFSET[3] = { 0.035204f, -0.164326f, 0.011369f };

// ── A matrix
──────────────────────────────────────────────────────────────────
const float A_D[3][3] = {
    { 0.99930577f, -0.00002068f, -0.00003493f },
    { 0.00233387f,  0.98443635f,  0.02453758f },
    {-0.00015568f,  0.00111512f,  0.99805077f }
};

// ── B matrix — col0=urine, col1=apple juice
──────────────────────────────
const float B_D[3][2] = {
    { 0.00000000f,  1.80650e-03f },
    { 6.84417e-04f, -4.73295e-03f },
    { 2.49130e-02f, -1.66300e-03f }
};

// ── LQR gain
──────────────────────────────────────────────────────────────────
const float K_LQR[2][3] = {
    {  0.52433f, -0.22060f,  27.80957f },
    {  7.94797f, -3.98169f, -176.98608f}
```

```
};

// ── Kalman: process noise Q_proc ──────────────────────────────────────────
const float Q_PROC[3] = { 0.010000f, 0.148808f, 0.001406f };

// ── Kalman: measurement noise R_meas ──────────────────────────────────────
const float R_MEAS[2] = { 0.157198f, 0.000532f };

// ── Kalman: error covariance P (3x3) ──────────────────────────────────────
float P_kal[3][3] = {
    { 1.0f, 0.0f, 0.0f },
    { 0.0f, 1.0f, 0.0f },
    { 0.0f, 0.0f, 1.0f }
};

// ── Observer state ────────────────────────────────────────────────────────
float x_obs[3] = { VFA0, EC0, PH0 };

// ── State machine ─────────────────────────────────────────────────────────
enum ExperimentPhase { ACID_1, LQR_RUN, ACID_2, BASELINE_RUN };
ExperimentPhase expPhase  = ACID_1;
int             expCycle  = 0;

unsigned long phaseStartMs  = 0;
unsigned long lastDoseMs    = 0;
unsigned long lastDoseMinMs = 0;
unsigned long lastStatusMs  = 0;
unsigned long lastCloudMs   = 0;
int           doseMinsElapsed = 0;

const unsigned long STATUS_INTERVAL_MS = 2000UL;
const unsigned long CLOUD_INTERVAL_MS  = 5000UL;

// ── Sensor cache ──────────────────────────────────────────────────────────
float latest_pH   = PH0, latest_EC  = EC0;
float latest_ORP  = 0;
float latest_Temp = 20.0f;
float latest_Q_uri = 0, latest_Q_aj = 0, latest_Q_pho = 0;
float latest_pH_V  = 0.0f, latest_EC_V  = 0.0f;
float latest_ORP_V = 0.0f;
bool  pH_valid = false;

//
==============================================================================
//  UTILITY
//
==============================================================================
float adcToVolts(int adc) { return (float)adc * (VREF / (float)ADC_MAX); }

int readADC_avg(int pin, int n = 30) {
    long sum = 0;
    for (int i = 0; i < n; i++) { sum += analogRead(pin); delay(2); }
    return (int)(sum / n);
}

float readPH() {
    long sum = 0;
```

```
    for (int i = 0; i < 30; i++) { sum += analogRead(PH_PIN_1); delay(2); }
    latest_pH_V = adcToVolts((int)(sum / 30));
    float ph = PH_M * latest_pH_V + PH_B;
    if (latest_pH_V >= 3.28f || latest_pH_V < 0.1f || ph < 0.0f || ph > 14.0f)
{
        pH_valid = false; return latest_pH;
    }
    pH_valid = true; return ph;
}

float readEC() {
    int raw = readADC_avg(EC_PIN_1);
    latest_EC_V = adcToVolts(raw);
    return EC_SLOPE_MSPERMV * latest_EC_V + EC_OFFSET_MS;
}

float readORP() {
    long sum = 0;
    for (int i = 0; i < 30; i++) { sum += analogRead(ORP_PIN_1); delay(2); }
    latest_ORP_V = adcToVolts((int)(sum / 30));
    return 1320.0f - latest_ORP_V * 1000.0f - ORP_OFFSET;
}

float readTemp() {
    tempBus1.requestTemperatures();
    float c = tempBus1.getTempCByIndex(0);
    if (c == DEVICE_DISCONNECTED_C || c == 85.0f || c < -55.0f || c > 125.0f)
return NAN;
    return c;
}

void doseML(int pin, float mL, float mL_per_sec, const char* name) {
    if (mL <= 0.0f) return;
    unsigned long ms = (unsigned long)((mL / mL_per_sec) * 1000.0f);
    if (ms < 30) ms = 30;
    Serial.print(F("  PUMP ")); Serial.print(name);
    Serial.print(F(": "));      Serial.print(mL, 3);
    Serial.print(F(" mL ("));   Serial.print(ms); Serial.println(F(" ms)"));
    digitalWrite(pin, HIGH); delay(ms); digitalWrite(pin, LOW); delay(100);
}

void resetP() {
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            P_kal[i][j] = (i == j) ? 1.0f : 0.0f;
}

//
==============================================================================
//  v22: COMPUTE AVERAGE URINE FRACTION FROM LQR LOGS
//  Called at the end of the LQR phase before transitioning to ACID_2.
//  Computes total urine / (total urine + total AJ) across all logged minutes.
//  Falls back to 0.5 if no data or degenerate.
//
==============================================================================
void computeBaselineRatio() {
    float total_uri = 0.0f, total_aj = 0.0f;
    for (int i = 0; i < flow_log_count; i++) {
        total_uri += lqr_uri_log[i];
        total_aj  += lqr_aj_log[i];
    }
    float grand_total = total_uri + total_aj;
    if (grand_total < 1e-6f || flow_log_count == 0) {
```

```
        baseline_uri_fraction = 0.5f;
        Serial.println(F("  [v22] No flow data — baseline ratio defaulting to
0.50"));
    } else {
        baseline_uri_fraction = total_uri / grand_total;
        Serial.print(F("  [v22] LQR totals: uri="));
        Serial.print(total_uri, 2);
        Serial.print(F(" mL  AJ="));
        Serial.print(total_aj, 2);
        Serial.print(F(" mL  => baseline_uri_fraction="));
        Serial.println(baseline_uri_fraction, 4);
    }
}

//
==============================================================================
//  KALMAN OBSERVER
//
==============================================================================
void observerUpdate(float u_uri, float u_aj, float meas_EC, float meas_pH) {
    const float x_nom[3] = { VFA0, EC0, PH0 };
    float u[2] = { u_uri, u_aj };

    // ── Predict
──────────────────────────────────────────────────────────────────
    float x_pred[3];
    for (int i = 0; i < 3; i++) {
        float ax = 0.0f, bu = 0.0f;
        for (int j = 0; j < 3; j++) ax += A_D[i][j] * (x_obs[j] - x_nom[j]);
        for (int j = 0; j < 2; j++) bu += B_D[i][j] * u[j];
        x_pred[i] = x_nom[i] + ax + bu + D_OFFSET[i];
    }

    float P_pred[3][3];
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            float s = 0.0f;
            for (int k = 0; k < 3; k++)
                for (int l = 0; l < 3; l++)
                    s += A_D[i][k] * P_kal[k][l] * A_D[j][l];
            P_pred[i][j] = s + (i == j ? Q_PROC[i] : 0.0f);
        }
    }

    // ── Innovation
──────────────────────────────────────────────────────────────────
    float innov[2] = {
        meas_EC - x_pred[1],
        meas_pH - x_pred[2]
    };

    // ── S = C*P_pred*C' + R  (C picks rows 1,2)
──────────────────────────────────────────────
    float S00 = P_pred[1][1] + R_MEAS[0];
    float S01 = P_pred[1][2];
    float S10 = P_pred[2][1];
    float S11 = P_pred[2][2] + R_MEAS[1];

    float det = S00 * S11 - S01 * S10;
    if (fabsf(det) < 1e-10f) det = 1e-10f;
    float Si00 =  S11 / det;
    float Si01 = -S01 / det;
    float Si10 = -S10 / det;
```

```
    float Si11 =  S00 / det;

    // ── K = P_pred*C'*S^-1
──────────────────────────────────────────────────
    float K[3][2];
    for (int i = 0; i < 3; i++) {
        float pc0 = P_pred[i][1];
        float pc1 = P_pred[i][2];
        K[i][0] = pc0 * Si00 + pc1 * Si10;
        K[i][1] = pc0 * Si01 + pc1 * Si11;
    }

    // ── State update
──────────────────────────────────────────────────────
    for (int i = 0; i < 3; i++)
        x_obs[i] = x_pred[i] + K[i][0] * innov[0] + K[i][1] * innov[1];

    x_obs[0] = constrain(x_obs[0], VFA_OBS_MIN, VFA_OBS_MAX);
    x_obs[1] = constrain(x_obs[1],  0.0f,  30.0f);
    x_obs[2] = constrain(x_obs[2],  3.0f,  12.0f);

    // ── Covariance update: P = (I - K*C) * P_pred
──────────────────────────
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            float IKC_ij = (i == j ? 1.0f : 0.0f)
                           - K[i][0] * (j == 1 ? 1.0f : 0.0f)
                           - K[i][1] * (j == 2 ? 1.0f : 0.0f);
            float s = 0.0f;
            for (int k = 0; k < 3; k++)
                s += IKC_ij * P_pred[i][k];
            P_kal[i][j] = s;
        }
    }
}

//
==============================================================================
//  LQR CONTROL LAW
//
==============================================================================
void lqrDose(float& u_uri, float& u_aj) {
    float dx[3] = { x_obs[0]-VFA0, latest_EC-EC0, latest_pH-PH0 };
    float raw_uri = U_BASE, raw_aj = U_BASE;
    for (int j = 0; j < 3; j++) {
        raw_uri -= K_LQR[0][j] * dx[j];
        raw_aj  -= K_LQR[1][j] * dx[j];
    }
    u_uri = constrain(raw_uri, U_MIN, U_MAX_URINE);
    u_aj  = constrain(raw_aj,  U_MIN, U_MAX_AJ);
}

//
==============================================================================
//  CLOUD SYNC
//
==============================================================================
void onCloudVFAMLChange() {}

void syncCloud(const char* lbl) {
    cloud_pH            = latest_pH;
    cloud_EC_mS         = latest_EC;
    cloud_ORP_mV        = latest_ORP;
```

```
    cloud_Temp_C        = latest_Temp;
    cloud_Q_urine_mL    = latest_Q_uri;
    cloud_Q_spir_mL     = latest_Q_aj;
    cloud_Q_kombucha_mL = latest_Q_pho;
    cloud_VFA_mL        = x_obs[0];
    cloud_stressState   = String(lbl);
    cloud_stressCycle   = expCycle;
    cloud_HRT           = 0;
    cloud_emergency     = false;
    cloud_holdRemaining_min = 0;
}

//
==============================================================================
//  SETUP
//
==============================================================================
void setup() {
    Serial.begin(115200); delay(1500);
    analogReadResolution(12);

    pinMode(PUMP_URINE_T1,      OUTPUT); digitalWrite(PUMP_URINE_T1,      LOW);
    pinMode(PUMP_AJ_T1,         OUTPUT); digitalWrite(PUMP_AJ_T1,         LOW);
    pinMode(PUMP_PHOSPHORIC_T1, OUTPUT); digitalWrite(PUMP_PHOSPHORIC_T1, LOW);
    pinMode(RELAY_UNUSED_1,     OUTPUT); digitalWrite(RELAY_UNUSED_1,     LOW);
    pinMode(RELAY_UNUSED_2,     OUTPUT); digitalWrite(RELAY_UNUSED_2,     LOW);

    tempBus1.begin();
    tempBus1.setWaitForConversion(true);
    tempBus1.setResolution(12);

    initProperties();
    ArduinoCloud.begin(ArduinoIoTPreferredConnection);
    setDebugMessageLevel(2);
    ArduinoCloud.printDebugInfo();

    unsigned long now = millis();
    phaseStartMs = lastDoseMs = lastDoseMinMs =
    lastStatusMs = lastCloudMs = now;

    latest_pH  = readPH();
    latest_EC  = readEC();
    latest_ORP = readORP();
    x_obs[0] = VFA0; x_obs[1] = latest_EC; x_obs[2] = latest_pH;
    resetP();

    expPhase        = ACID_1;
    flow_log_count  = 0;
    doseMinsElapsed = 0;
    baseline_uri_fraction = 0.5f;


Serial.println(F("=========================================================="));
    Serial.println(F("  3-STATE LQR + KALMAN OBSERVER vs BASELINE  —  v22"));
    Serial.println(F("  BASELINE: fixed ratio from LQR average (not 50:50)"));
    Serial.println(F("  MFC on separate Arduino (Chickie)"));
    Serial.println(F("  Acid phases: 3 mL/min phosphoric until pH <= 6.5"));
    Serial.println(F("  Kalman: Q=diag([0.01,0.1488,0.0014])
R=diag([0.1572,0.0005])"));
    Serial.println(F("  LQR:    Q=diag([0.01,0.01,1000])
R=diag([0.01,0.0001])"));
    Serial.print  (F("  Setpoints: VFA=")); Serial.print(VFA0,2);
    Serial.print  (F("  EC=")); Serial.print(EC0,2);
```

```
    Serial.print  (F("  pH="));            Serial.println(PH0,2);

Serial.println(F("=========================================================="));

Serial.println(F("CSVHEADER,Time_s,Phase,Cycle,pH,EC_mS,ORP_mV,Temp_C,Q_uri,Q_a
j,Q_pho,VFA_est,EC_obs,pH_obs,pH_raw_V,EC_raw_V,ORP_raw_V,pH_valid,P00,P11,P22"
));
    Serial.println(F("=== STARTING ACID_1 ==="));
}

//
==============================================================================
//  LOOP
//
==============================================================================
void loop() {
    ArduinoCloud.update();
    digitalWrite(RELAY_UNUSED_1, LOW);
    digitalWrite(RELAY_UNUSED_2, LOW);
    unsigned long now = millis();

    // ── Sensor read + serial log
──────────────────────────────────────────────
    if (now - lastStatusMs >= STATUS_INTERVAL_MS) {
        lastStatusMs = now;
        float EC = readEC(); float Tmp = readTemp();
        latest_EC  = EC;
        latest_pH  = readPH();
        latest_ORP = readORP();
        if (!isnan(Tmp)) latest_Temp = Tmp;

        const char* lbl = (expPhase==ACID_1)   ? "ACID_1"   :
                          (expPhase==LQR_RUN)   ? "LQR"      :
                          (expPhase==ACID_2)    ? "ACID_2"   : "BASELINE";

        Serial.print(F("DATA,"));  Serial.print(now/1000);
        Serial.print(F(","));      Serial.print(lbl);
        Serial.print(F(","));      Serial.print(expCycle);
        Serial.print(F(","));      Serial.print(latest_pH,    3);
        Serial.print(F(","));      Serial.print(latest_EC,    3);
        Serial.print(F(","));      Serial.print(latest_ORP,   1);
        Serial.print(F(","));
        if (isnan(Tmp)) Serial.print(F("NA")); else Serial.print(Tmp, 2);
        Serial.print(F(","));      Serial.print(latest_Q_uri, 4);
        Serial.print(F(","));      Serial.print(latest_Q_aj,  4);
        Serial.print(F(","));      Serial.print(latest_Q_pho, 4);
        Serial.print(F(","));      Serial.print(x_obs[0],     4);
        Serial.print(F(","));      Serial.print(x_obs[1],     4);
        Serial.print(F(","));      Serial.print(x_obs[2],     4);
        Serial.print(F(","));      Serial.print(latest_pH_V,  4);
        Serial.print(F(","));      Serial.print(latest_EC_V,  4);
        Serial.print(F(","));      Serial.print(latest_ORP_V, 4);
        Serial.print(F(","));      Serial.print(pH_valid ? F("1") : F("0"));
        Serial.print(F(","));      Serial.print(P_kal[0][0],  5);
        Serial.print(F(","));      Serial.print(P_kal[1][1],  5);
        Serial.print(F(","));      Serial.println(P_kal[2][2],5);
    }

    // ── Cloud sync
──────────────────────────────────────────────────────────
    if (now - lastCloudMs >= CLOUD_INTERVAL_MS) {
        const char* lbl = (expPhase==ACID_1)   ? "ACID_1"   :
                          (expPhase==LQR_RUN)   ? "LQR"      :
```

```
                        (expPhase==ACID_2)    ? "ACID_2"   : "BASELINE";
        syncCloud(lbl); lastCloudMs = now;
    }

    // ── ACID PHASES (1 & 2)
──────────────────────────────────────────────────────
    if (expPhase == ACID_1 || expPhase == ACID_2) {

        if (latest_pH <= PH_LOWER_BOUND && pH_valid) {
            latest_Q_pho = 0.0f;
            if (expPhase == ACID_1) {
                expPhase = LQR_RUN;
                flow_log_count  = 0;
                doseMinsElapsed = 0;
                phaseStartMs = lastDoseMinMs = now;
                x_obs[0] = VFA0; x_obs[1] = latest_EC; x_obs[2] = latest_pH;
                resetP();
                Serial.println(F("=== pH <= 6.50 -> STARTING LQR RUN (2h)
===="));
            } else {
                expCycle++;
                expPhase = BASELINE_RUN;
                doseMinsElapsed = 0;
                phaseStartMs = lastDoseMinMs = now;
                x_obs[0] = VFA0; x_obs[1] = latest_EC; x_obs[2] = latest_pH;
                resetP();
                Serial.print(F("=== pH <= 6.50 -> STARTING BASELINE RUN (2h)
cycle="));
                Serial.print(expCycle);
                Serial.print(F("  uri_fraction="));
                Serial.println(baseline_uri_fraction, 4);
            }
            return;
        }

        if (now - lastDoseMs >= PHOS_INTERVAL_MS) {
            lastDoseMs = now;
            latest_Q_pho = DOSE_ML_PHOS;
            latest_Q_uri = 0.0f; latest_Q_aj = 0.0f;
            doseML(PUMP_PHOSPHORIC_T1, DOSE_ML_PHOS, ML_PER_SEC_PHOSPHORIC,
"PHOSPHORIC");
            observerUpdate(0.0f, 0.0f, latest_EC, latest_pH);
            Serial.print(F("  [ACID] pH="));   Serial.print(latest_pH, 3);
            Serial.print(F("  valid="));        Serial.print(pH_valid ?
F("Y") : F("N"));
            Serial.print(F("  dose_mL="));      Serial.print(DOSE_ML_PHOS, 1);
            Serial.print(F("  target="));       Serial.println(PH_LOWER_BOUND,
2);
        }
        return;
    }

    // ── LQR RUN (2 hours)
──────────────────────────────────────────────────────
    if (expPhase == LQR_RUN) {
        if (now - phaseStartMs >= RUN_DURATION_MS) {
            // v22: compute average ratio before transitioning
            computeBaselineRatio();
            expPhase     = ACID_2;
            phaseStartMs = lastDoseMs = now;
            latest_Q_uri = latest_Q_aj = 0.0f;
            Serial.println(F("=== LQR 2h complete -> ACID_2 ===="));
            return;
```

```
        }
        if (now - lastDoseMinMs >= 60000UL) {
            lastDoseMinMs = now;
            float u_uri, u_aj;
            lqrDose(u_uri, u_aj);

            // v22: log urine and AJ separately as well as total
            if (flow_log_count < MAX_FLOW_LOG) {
                lqr_flow_log[flow_log_count] = u_uri + u_aj;
                lqr_uri_log[flow_log_count]  = u_uri;
                lqr_aj_log[flow_log_count]   = u_aj;
                flow_log_count++;
            }

            latest_Q_uri = u_uri; latest_Q_aj = u_aj; latest_Q_pho = 0.0f;
            doseML(PUMP_URINE_T1, u_uri, ML_PER_SEC_URINE, "URINE");
            doseML(PUMP_AJ_T1,   u_aj,  ML_PER_SEC_AJ,    "APPLE_JUICE");
            doseMinsElapsed++;
            observerUpdate(u_uri, u_aj, latest_EC, latest_pH);
            Serial.print(F("  [LQR] min=")); Serial.print(doseMinsElapsed);
            Serial.print(F("  pH=")); Serial.print(latest_pH,  3);
            Serial.print(F("  valid=")); Serial.print(pH_valid ? F("Y") :
F("N"));
            Serial.print(F("  EC=")); Serial.print(latest_EC,   3);
            Serial.print(F("  VFA_est=")); Serial.print(x_obs[0],    4);
            Serial.print(F("  u_uri=")); Serial.print(u_uri,       4);
            Serial.print(F("  u_aj=")); Serial.print(u_aj,        4);
            Serial.print(F("  P00=")); Serial.print(P_kal[0][0], 5);
            Serial.print(F("  P11=")); Serial.print(P_kal[1][1], 5);
            Serial.print(F("  P22=")); Serial.println(P_kal[2][2],5);
            syncCloud("LQR");
        }
        return;
    }

    // —— BASELINE RUN (2 hours)
————————————————————————————————————————————————

    if (expPhase == BASELINE_RUN) {
        if (now - phaseStartMs >= RUN_DURATION_MS) {
            expPhase     = ACID_1;
            phaseStartMs = lastDoseMs = now;
            latest_Q_uri = latest_Q_aj = 0.0f;
            Serial.println(F("=== BASELINE 2h complete -> ACID_1 ===")); 
            return;
        }
        if (now - lastDoseMinMs >= 60000UL) {
            lastDoseMinMs = now;

            // Total flow for this minute from LQR log (same as v21)
            float total = 0.0f;
            if      (doseMinsElapsed < flow_log_count) total =
lqr_flow_log[doseMinsElapsed];
            else if (flow_log_count > 0)               total =
lqr_flow_log[flow_log_count-1];

            // v22: split using fixed ratio from LQR average instead of 50:50
            float u_uri = constrain(total * baseline_uri_fraction,       U_MIN,
U_MAX_URINE);
            float u_aj  = constrain(total * (1.0f - baseline_uri_fraction),
U_MIN, U_MAX_AJ);

            latest_Q_uri = u_uri; latest_Q_aj = u_aj; latest_Q_pho = 0.0f;
            doseML(PUMP_URINE_T1, u_uri, ML_PER_SEC_URINE, "URINE");
```

```
            doseML(PUMP_AJ_T1,   u_aj,  ML_PER_SEC_AJ,    "APPLE_JUICE");
            doseMinsElapsed++;
            observerUpdate(u_uri, u_aj, latest_EC, latest_pH);
            Serial.print(F("  [BASE] min="));
Serial.print(doseMinsElapsed);
            Serial.print(F("  pH="));                 Serial.print(latest_pH,
3);
            Serial.print(F("  EC="));                 Serial.print(latest_EC,
3);
            Serial.print(F("  VFA_est="));            Serial.print(x_obs[0],
4);
            Serial.print(F("  total_mL="));           Serial.print(total,
4);
            Serial.print(F("  uri_frac="));
Serial.print(baseline_uri_fraction, 4);
            Serial.print(F("  u_uri="));              Serial.print(u_uri,
4);
            Serial.print(F("  u_aj="));               Serial.println(u_aj,
4);
            syncCloud("BASELINE");
        }
        return;
    }
}
```

Python Analysis Code

The following Python code was used to load Arduino IoT Cloud CSV exports, identify LQR and baseline phases from the stressState variable, compute mean and integrated metrics for each phase, and generate the summary tables and figures reported in Chapter 4. The code requires Python 3.11 with pandas 2.0 and NumPy 1.25 installed.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import differential_evolution, minimize


# ── Load data ──────────────────────────────────────────────────────────
def load_cloud_csv(path):
    df = pd.read_csv(path, parse_dates=['time'])
    df['time'] = pd.to_datetime(df['time'], utc=True)
    df = df.sort_values('time').reset_index(drop=True)
    return df


mfc  = load_cloud_csv('Digester_Tank_1-cloud_MFC_mV.csv')
orp  = load_cloud_csv('Digester_Tank_1-cloud_ORP_mV.csv')
ph   = load_cloud_csv('Digester_Tank_1-cloud_pH.csv')
ec   = load_cloud_csv('Digester_Tank_1-cloud_EC_mS.csv')
state = load_cloud_csv('Digester_Tank_1-cloud_stressState.csv')
cycle = load_cloud_csv('Digester_Tank_1-cloud_stressCycle.csv')
```

```
# ── Identify phase windows ──────────────────────────────────────
def get_phase_windows(state_df, target_state):
    windows = []
    in_phase = False
    start = None
    for _, row in state_df.iterrows():
        if row['value'] == target_state and not in_phase:
            in_phase = True
            start = row['time']
        elif row['value'] != target_state and in_phase:
            in_phase = False
            windows.append((start, row['time']))
    if in_phase:
        windows.append((start, state_df['time'].iloc[-1]))
    return windows



lqr_windows      = get_phase_windows(state, 'LQR')
baseline_windows = get_phase_windows(state, 'BASELINE')



# ── Compute metrics for a window ────────────────────────────────
def window_metrics(df, t_start, t_end, value_col='value'):
    mask = (df['time'] >= t_start) & (df['time'] <= t_end)
    sub = df[mask].copy()
    if len(sub) < 2:
        return {'mean': np.nan, 'integrated': np.nan}
    t_min = (sub['time'] - t_start).dt.total_seconds() / 60.0
    mean_val = sub[value_col].mean()
    integrated = np.trapz(sub[value_col].values, t_min.values)
    return {'mean': mean_val, 'integrated': integrated}



# ── Compare matched LQR and baseline windows ────────────────────
print('\n=== EXPERIMENT 2 RESULTS ===')
for i, (lqr_win, bl_win) in enumerate(zip(lqr_windows, baseline_windows)):
    lqr_mfc = window_metrics(mfc, *lqr_win)
    bl_mfc  = window_metrics(mfc, *bl_win)
    lqr_orp = window_metrics(orp, *lqr_win)
    bl_orp  = window_metrics(orp, *bl_win)
    lqr_ph  = window_metrics(ph,  *lqr_win)
    bl_ph   = window_metrics(ph,  *bl_win)
    lqr_ec  = window_metrics(ec,  *lqr_win)
    bl_ec   = window_metrics(ec,  *bl_win)
    pct = (lqr_mfc['integrated'] - bl_mfc['integrated']) / bl_mfc['integrated']
* 100
    print(f'\nCycle {i+1}:')
    print(f'  MFC: LQR={lqr_mfc["integrated"]:.0f}
BL={bl_mfc["integrated"]:.0f}  +{pct:.1f}%')
    print(f'  ORP: LQR={lqr_orp["mean"]:.1f}  BL={bl_orp["mean"]:.1f}')
    print(f'  pH:  LQR={lqr_ph["mean"]:.3f}  BL={bl_ph["mean"]:.3f}')
    print(f'  EC:  LQR={lqr_ec["mean"]:.3f}  BL={bl_ec["mean"]:.3f}')



# ── System identification (B matrix, Nelder-Mead) ───────────────
def simulate_one_step(A, B, D, x, u, x_sp):
    dx = x - x_sp
    return A @ dx + x_sp + B @ u + D
```

```
def objective(params, data, x_sp, D, A, param_idx):
    B = np.zeros((3, 2))
    for i, idx in enumerate(param_idx):
        B[idx[0], idx[1]] = params[i]
    errors = []
    for t in range(1, len(data)):
        x_prev = data[t-1, :3]
        u_prev = data[t-1, 3:5]
        x_pred = simulate_one_step(A, B, D, x_prev, u_prev, x_sp)
        errors.append((x_pred[1] - data[t, 1])**2 +
                      (x_pred[2] - data[t, 2])**2)
    return np.sum(errors)


# ── pH time series plot ──────────────────────────────────────────────
fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(ph['time'], ph['value'], color='cyan', lw=1, label='pH')
ax.axhline(7.2, color='gold', lw=1.5, ls='--', label='Setpoint 7.2')
for w in lqr_windows:
    ax.axvspan(w[0], w[1], alpha=0.15, color='blue', label='LQR')
for w in baseline_windows:
    ax.axvspan(w[0], w[1], alpha=0.15, color='red', label='Baseline')
ax.set_xlabel('Date/Time (EST)')
ax.set_ylabel('pH')
ax.set_title('Digester Tank 1 -- pH')
ax.legend(loc='upper right')
plt.tight_layout()
plt.savefig('ph_timeseries.png', dpi=150)
```